

Go (Golang)

SDE2 Interview Guide

100 Questions & Answers with Code Examples

Level: Mid (2–4 yrs)	Stack: Go / Golang	Roles: SDE2 / Backend
----------------------	--------------------	-----------------------

#	Topic	Questions
1	Goroutines & Concurrency	Q1–Q15
2	Channels & Select	Q16–Q28
3	Runtime & Memory	Q29–Q40
4	Interfaces & Types	Q41–Q52
5	Error Handling	Q53–Q60
6	Context	Q61–Q67
7	Generics	Q68–Q74
8	Testing & Benchmarking	Q75–Q82
9	Go Patterns	Q83–Q92
10	Miscellaneous & Advanced	Q93–Q100

> **Focus:** Kafka Internals, RabbitMQ AMQP, Messaging Patterns, Go Integration | **Level:** SDE2

Table of Contents

[Kafka Advanced Internals](#2-kafka-advanced-internals) — Q26–Q50
 [RabbitMQ Fundamentals](#3-rabbitmq-fundamentals) — Q51–Q70
 [Messaging Patterns & Design](#4-messaging-patterns--design) — Q71–Q85
 [Go Integration](#5-go-integration) — Q86–Q100

Kafka Fundamentals

Q1. What is Apache Kafka and what makes it different from a traditional message queue?

Difficulty: Easy

Kafka is a **distributed append-only log** (not a traditional queue). Messages are retained after consumption (configurable duration). Multiple consumers can independently read the same messages at different offsets. Designed for high throughput (millions/sec) and horizontal scale.

Traditional Queue: Message deleted after consumption

Kafka Log: [msg1][msg2][msg3][msg4]...

 ↑ ↑

 Consumer A Consumer B (independent)

 offset=2 offset=4

Key differences: retention, replay, ordering per partition, consumer groups, sequential disk I/O, horizontal scale.

Q2. What are topics, partitions, and offsets?

Difficulty: Easy

Topic: named, append-only log (like a table)

Partition: one ordered, immutable segment of a topic

Topic "orders" → partition-0, partition-1, partition-2

Offset: position of a message within a partition (0, 1, 2, ...)

Segment: partition is split into segment files on disk

orders (topic)

■■■ partition-0: [msg0][msg1][msg2]... offset 0,1,2

■■■ partition-1: [msg0][msg1][msg3]... independent sequence

■■■ partition-2: [msg0][msg1]...

Key → $\text{hash}(\text{key}) \% N$ → determines partition

No key → round-robin across partitions

Same key → same partition (ordering guarantee)

Q3. What is a consumer group?

Difficulty: Easy

Consumer group: N consumers sharing partition consumption

Each partition assigned to exactly ONE consumer in the group

Different groups each get ALL messages independently

orders (3 partitions)

■■■ p0 → Consumer A (group1)

■■■ p1 → Consumer B (group1)

■■■ p2 → Consumer C (group1)

orders (same topic)

```
■■■ p0 → Consumer X (group2) ← gets same messages as A
■■■ p1 → Consumer Y (group2)
■■■ p2 → Consumer Z (group2)
```

Rule: consumers in group $\leq$ partitions

If consumers > partitions: extra consumers are idle

Scale: add partitions → can add consumers

Q4. What is Kafka's storage architecture?

Difficulty:
Medium

Broker storage structure:

```
/kafka-logs/
orders-0/          ← partition directory
  000000000000.log   ← segment file (messages)
  000000000000.index ← offset → file position index
  000000000000.timeindex ← timestamp → offset index
orders-1/
...
```

Segment: 1 GB by default (log.segment.bytes)

Index: sparse (not every offset), binary search to find position

Write: always append to active segment (sequential I/O → fast)

Read: seek to offset via index, read from segment file

Retention:

Time: log.retention.hours = 168 (7 days)

Size: log.retention.bytes = -1 (unlimited)

Compaction: keep latest value per key (like key-value store)

Q5. What is the role of ZooKeeper / KRaft in Kafka?

Difficulty:
Medium

Old (< Kafka 2.8): ZooKeeper manages cluster metadata

- Controller election
- Broker registration
- Topic/partition metadata
- Consumer group offsets (old API)

Problems: ZooKeeper is operational complexity,
limits scalability to ~hundreds of thousands of partitions

KRaft (Kafka Raft, $\geq$ Kafka 3.3 production-ready):

- Metadata stored in Kafka itself (internal __cluster_metadata topic)
- Controller quorum using Raft consensus
- No ZooKeeper dependency
- Supports millions of partitions
- Simpler deployment

Kafka 4.0: ZooKeeper mode removed entirely

Q6. What is replication in Kafka?

Difficulty:
Medium

replication.factor = 3: each partition has 1 leader + 2 followers

Leader: handles all reads and writes

Followers: replicate from leader

ISR (In-Sync Replicas): followers caught up to leader
Leader sends to all ISR before ack to producer

acks setting:

acks=0: fire and forget (possible data loss)
acks=1: leader writes (follower may not have it → loss on leader crash)
acks=all / acks=-1: all ISR write → no data loss (safest)

min.insync.replicas=2: minimum replicas that must ack
If ISR < min.insync.replicas: producer gets error (safety valve)

Leader election: if leader fails, Kafka controller elects ISR follower
Unclean election: allow non-ISR follower? (data loss vs availability)

Q7. What are producer delivery semantics?

Difficulty:
Medium

At-most-once (acks=0):

Send and forget → messages may be lost
Fastest, for non-critical metrics/logs

At-least-once (acks=all, retries > 0):

Producer retries on failure → possible duplicates
Consumer must be idempotent
Default for most production use

Exactly-once (idempotent producer + transactional):

enable.idempotence = true
→ Kafka assigns sequence numbers, deduplicates within session

Transactional producer:

BEGIN_TRANSACTION → produce to multiple partitions → COMMIT
Consumer: isolation.level = read_committed
→ atomically write to multiple topics/partitions, no duplicates

```
producer.initTransactions()
producer.beginTransaction()
producer.send("orders", msg)
producer.send("inventory", msg)
producer.commitTransaction() // atomic
```

Q8. What are consumer delivery semantics?

Difficulty:
Medium

At-most-once:

Commit offset BEFORE processing
If crash after commit but before process → message lost
enable.auto.commit = true, auto.commit.interval.ms = 5000

At-least-once (most common):

Process THEN commit offset
If crash after process but before commit → message reprocessed
Consumer must be idempotent!
enable.auto.commit = false (manual commit after processing)

Exactly-once:

Option 1: Transactional producer → write output + commit offset atomically
Option 2: Idempotency key in processing + dedup in consumer

Option 3: Kafka Streams exactly-once semantics (processing.guarantee=exactly_once_v2)

Q9. What is consumer offset management?

Difficulty:
Medium

Offsets stored in `__consumer_offsets` topic (Kafka internal)

Commit strategies:

Auto-commit: `enable.auto.commit=true` (risky: commits before processing done)
 Manual sync: `commitSync()` after each batch (slow but safe)
 Manual async: `commitAsync()` (fast, no retry on failure)
 Per-message: `commitSync({partition: offset+1})` for exactly-once semantics

Go (sarama/franz-go):

```
session.MarkMessage(msg, "") // mark for commit
session.Commit()             // commit marked offsets
```

Offset reset (no committed offset or offset out of range):

```
auto.offset.reset = earliest // read from beginning
auto.offset.reset = latest   // read only new messages (default)
auto.offset.reset = none     // throw error
```

Reset committed offset manually:

```
kafka-consumer-groups.sh --reset-offsets --to-earliest --group mygroup --topic orders
```

Q10. What is consumer group rebalancing?

Difficulty: Hard

Rebalance: redistribution of partitions among consumers in a group

Triggered by: consumer joins/leaves, partition added, heartbeat timeout

Protocol:

1. Consumer stops polling → triggers `session.timeout.ms` (default 45s)
2. Group coordinator (broker) detects → revokes all partition assignments
3. All consumers in group re-join
4. Leader (first joiner) assigns partitions using `partition.assignment.strategy`
5. All consumers receive new assignment

Partition assignment strategies:

RangeAssignor (default): ranges of partitions per consumer
 RoundRobinAssignor: round-robin across consumers
 StickyAssignor: minimize partition movement (keep previous assignments)
 CooperativeStickyAssignor: incremental rebalance (only move necessary partitions)
 → No stop-the-world; consumers keep processing unchanged partitions

Impact: during rebalance, all consumers stop processing (unless Cooperative)

Fix: use `CooperativeStickyAssignor`, tune `session.timeout.ms` + `heartbeat.interval.ms`

Q11. What are key Kafka producer configuration parameters?

Difficulty:
Medium

Reliability

```
acks = all                # wait for all ISR
retries = 2147483647       # retry until delivery.timeout.ms
enable.idempotence = true  # dedup within session
max.in.flight.requests.per.connection = 5 # 1 if strict ordering without idempotence
```

Performance

```
batch.size = 16384          # bytes per batch (increase for throughput)
linger.ms = 5                # wait up to 5ms to fill batch
compression.type = snappy    # lz4 or zstd for better compression
buffer.memory = 33554432     # 32MB producer buffer

# Timeouts
delivery.timeout.ms = 120000 # total time to deliver a message
request.timeout.ms = 30000   # per-request timeout

# Keys
key.serializer = StringSerializer
value.serializer = StringSerializer # or AvroSerializer
```

Q12. What are key Kafka consumer configuration parameters?

Difficulty:
Medium

```
# Consumer group
group.id = my-service
auto.offset.reset = earliest    # or latest
enable.auto.commit = false     # manual commits for reliability

# Poll tuning
max.poll.records = 500         # max records per poll()
max.poll.interval.ms = 300000  # max time between polls before rebalance
fetch.min.bytes = 1            # wait for this many bytes
fetch.max.wait.ms = 500        # max wait if fetch.min.bytes not met

# Session / heartbeat
session.timeout.ms = 45000     # detection of dead consumer
heartbeat.interval.ms = 15000  # heartbeat frequency (< session timeout/3)

# Deserialization
key.deserializer = StringDeserializer
value.deserializer = StringDeserializer # or AvroDeserializer
```

Q13. What is the Kafka leader and controller?

Difficulty:
Medium

Leader: for each partition, one broker is the leader

- all reads and writes go to the leader
- followers replicate from leader

Controller: one broker per cluster (elected by ZooKeeper/KRaft)

- manages leader elections (when broker dies)
- manages partition assignment
- handles broker joins/failures
- NOT in the data path

Broker failure:

- Controller detects missing heartbeat
- Controller selects new leader from ISR
- Updates metadata → brokers cache
- Producers/consumers detect via metadata update → reconnect to new leader

Typical failover time: 30-60 seconds (ZooKeeper) or < 10s (KRaft)

Q14. What is Kafka log compaction?**Difficulty: Hard**

Compaction: keep only the latest value per key
 vs Retention: keep all messages for N days/GB

Use case: user profiles, product catalog, configuration – only latest matters
 Compacted topic: behaves like a persistent key-value store

How it works:

Cleaner threads scan log segments
 For each key: keep latest message, delete older ones
 Tombstone: message with null value → delete key from compacted log

Configuration:

```
cleanup.policy = compact          # or delete (default) or compact,delete
min.compaction.lag.ms = 0        # don't compact messages newer than X
delete.retention.ms = 86400000   # tombstones kept for 1 day
```

Use case:

Source-of-truth topics: final state is what matters
 Kafka → consumer → materialized view (always up-to-date)
 DB changelog: keeps latest row per primary key

Q15. What is exactly-once semantics (EOS) in Kafka?**Difficulty: Hard**

`EOS = idempotent producer + transactional API + read_committed consumers`

Idempotent producer:

`enable.idempotence = true`
 Kafka assigns (ProducerID, SequenceNumber) per partition
 Broker deduplicates retries with same (PID, SeqNum)
 Does NOT protect across producer restarts (new PID each time)

Transactional producer:

Atomic write across multiple partitions/topics
 Offset commit included in transaction → EOS for consume-transform-produce

```
producer.initTransactions()
producer.beginTransaction()
for msg in messages:
    producer.send(output_topic, transform(msg))
producer.sendOffsetsToTransaction(offsets, consumerGroupMetadata)
producer.commitTransaction()
```

Consumers:

`isolation.level = read_committed` # skip uncommitted (aborted) messages

Kafka Streams: `processing.guarantee = exactly_once_v2`
 Automatically handles EOS for stream processing

Q16. What are Kafka topics best practices?**Difficulty: Medium**

Partition count:

= max consumers in group (for parallelism)
 = `throughput_target / throughput_per_partition`

Rule: start with 6, increase if needed (partitions can be added, not removed easily)

Replication factor:

Production: 3 (tolerate 1 broker failure)

Dev: 1

Naming conventions:

<domain>.<entity>.<action>

e.g.: payments.orders.created

users.profiles.updated

Retention:

Default: 7 days / no size limit

Event sourcing: set large (months/years)

Metrics: 1-2 days

Compaction for:

Current state (profiles, configs, last-known-value)

Separate topics for:

- Different consumer groups with different retention needs
- Different throughput requirements
- Different message schemas

Q17. What is Kafka Streams?

Difficulty: Hard

Kafka Streams: Java library for stream processing directly on Kafka

No separate cluster needed (runs in application)

Concepts:

KStream: unbounded stream of events (all records)

KTable: materialized view of stream (latest per key = like a DB table)

GlobalKTable: KTable replicated to ALL instances (for enrichment)

Operations:

filter, map, flatMap, groupBy, aggregate, join, windowing

Join types:

Stream-Stream: within time window

Stream-Table: enrich stream with table lookup

Table-Table: merge two tables

Windowing:

Tumbling: fixed, non-overlapping (every 5 min)

Hopping: fixed, overlapping (every 1 min, 5 min wide)

Session: event-driven (gap-based)

State stores:

RocksDB embedded in each instance

Changelog topic in Kafka for durability/recovery

// Wordcount example

```
KStream<String, String> source = builder.stream("text-input");
source.flatMapValues(v -> Arrays.asList(v.split(" ")))
    .groupBy((k, word) -> word)
    .count()
    .toStream()
```



```
.to("word-count-output");
```

Q18. What is the difference between Kafka and a traditional queue like SQS?

Difficulty: Easy

	Kafka	SQS
Retention	Configurable (days/forever)	Until consumed (or TTL)
Replay	Yes (any offset)	No
Ordering	Per partition	Best-effort (FIFO queues: per group)
Scale	Millions msg/sec	Thousands/sec
Consumers	Multiple independent groups	One or many (compete)
Infra	Self-hosted (or cloud)	Fully managed AWS
Cost	Infrastructure cost	Pay per use (cheap at low volume)
Use case	Event sourcing, streams	Simple task queues, decoupling

When to choose Kafka:

- Multiple services need same events independently
- Replay needed (audit, reprocessing)
- High throughput (>100K msg/sec)
- Event sourcing, CDC, stream analytics

When to choose SQS:

- Simple decoupling of one producer → one consumer
- AWS-native integration
- Low volume, want zero infrastructure
- Standard task queue

Q19. What is consumer lag and how do you monitor it?

Difficulty: Easy

Consumer lag: difference between latest offset and consumer committed offset
= number of messages not yet processed

Latest offset: 1000 (newest message in partition)

Committed offset: 800 (consumer processed up to here)

Lag = 200 messages

Monitor with:

```
kafka-consumer-groups.sh --describe --group my-group
```

Shows: TOPIC, PARTITION, CURRENT-OFFSET, LOG-END-OFFSET, LAG

Metrics (via JMX, Prometheus):

records-lag (per partition)

records-lag-max (worst partition)

records-consumed-rate

fetch-rate

Alerts:

lag > 10000: consumer can't keep up

lag growing: scale consumers or optimize processing

lag = 0: consumer is healthy

Auto-scaling: Kafka-based autoscaler (KEDA) scales consumer pods based on lag

Q20. What is Kafka Connect?

Difficulty:
Medium

Kafka Connect: framework for streaming data between Kafka and external systems

Source Connector: external system → Kafka
 Debezium (PostgreSQL CDC → Kafka)
 S3 Source, JDBC Source, Elasticsearch Source

Sink Connector: Kafka → external system
 S3 Sink (archive to S3)
 Elasticsearch Sink (search index)
 JDBC Sink (write to DB)
 BigQuery Sink

Advantages:

- No code: just configuration (JSON)
- Exactly-once for supported connectors
- Distributed (workers scale horizontally)
- Automatic offset management

Example: Debezium PostgreSQL connector

- Reads PostgreSQL WAL → publishes row changes to Kafka
- Topic: mydb.public.orders → every INSERT/UPDATE/DELETE
- Payload: before/after row images

Start connector

```
curl -X POST /connectors -d @debezium-config.json
```

Q21. What is Kafka Schema Registry?

Difficulty:
Medium

Schema Registry: stores and manages Avro/Protobuf/JSON schemas
 Enables schema evolution without breaking producers/consumers

How it works:

- Producer: serialize with schema → register schema → send (schema_id + payload)
- Consumer: receive → extract schema_id → fetch schema → deserialize

Schema evolution rules:

- BACKWARD: new schema can read old data (add optional fields)
- FORWARD: old schema can read new data (remove optional fields)
- FULL: both backward and forward compatible
- NONE: no compatibility check

Benefits:

- Small messages (schema ID = 4 bytes, not full schema embedded)
- Type safety between services
- Enforced schema contract
- Multiple versions tracked

// Go (confluent-kafka-go with Avro):

```
client, _ := schemaregistry.NewClient(schemaregistry.NewConfig("http://schema-registry:8081"))
ser, _ := avro.NewSerializer(client, serde.ValueSerde, avro.NewSerializerConfig())
payload, _ := ser.Serialize("orders", &Order{...})
```

Q22. What is Kafka's throughput capability?

Difficulty: Easy

Why Kafka is fast:

1. Sequential disk I/O: writes always append to log end
 Sequential I/O: 600 MB/s vs Random I/O: 50 MB/s

2. Zero-copy: `sendfile()` syscall
Disk → kernel buffer → network (skips userspace copy)
2-4x fewer CPU cycles per message
3. Batching: producer batches multiple messages per request
Compression per batch (snappy/lz4/zstd)
Fewer network round-trips
4. OS page cache: Kafka doesn't manage memory itself
Relies on OS page cache for hot data
Works well: consumers usually read recent messages (in cache)

Performance:

Single broker: ~700K messages/sec write, ~1M read
 With 3 brokers + 10 partitions: millions/sec
 LinkedIn: originally designed for 100K+ msg/sec per server

Q23. What is the difference between pull and push in Kafka?

Difficulty: Easy

Kafka uses PULL (consumer polls broker):
`consumer.poll(Duration.ofMillis(100))`

Benefits of pull:

Consumer controls rate (no overwhelming)
 Consumer can batch (wait for `min.bytes`)
 Consumer can pause/resume
 No flow control needed from broker side

vs Push (broker pushes):

Lower latency potential
 Simpler consumer (no polling loop)
 But: broker must track consumer capacity
 Risk: overwhelming slow consumers

Kafka's pull design:

`fetch.min.bytes = 1024` → wait until 1KB available
`fetch.max.wait.ms = 500` → max wait time
`max.poll.records = 500` → max messages per poll

Q24. What is Kafka's exactly-once delivery for stream processing?

Difficulty: Hard

Stream processing: consume → transform → produce

Problem: if crash after produce but before offset commit:
 → reprocess → duplicate output messages

Solution (Kafka EOS):

1. Consumer reads messages
2. Transforms them
3. Produces output + commits input offsets atomically in ONE transaction

```
producer.beginTransaction()
for record in consumer.poll():
    output = transform(record.value())
    producer.send(output_topic, output)
producer.sendOffsetsToTransaction(offsets, consumer.groupMetadata())
```

```
producer.commitTransaction() // atomic: output + offset commit
```

If commit fails: transaction aborted

- consumer reprocesses from last committed offset
- output messages discarded (read_committed consumers skip them)
- net result: exactly-once processing

Kafka Streams: `processing.guarantee=exactly_once_v2` handles this automatically

Q25. What is Kafka MirrorMaker 2?

Difficulty:
Medium

MirrorMaker 2: replicate data between Kafka clusters

Use case: geo-replication, disaster recovery, cloud migration

Architecture:

- Source cluster → MirrorMaker 2 → Destination cluster
- Topics: `source.orders` → `dest: source-cluster.orders` (prefix)
- Offsets translated (source offsets ≠ destination offsets)
- Consumer groups replicated with offset translation

Active-Active: both clusters accept writes, MM2 syncs bidirectionally

Active-Passive: primary → secondary (DR)

Configuration:

- `clusters = source, destination`
- `source→destination.enabled = true`
- `source→destination.topics = .*` # all topics
- `source→destination.groups = .*` # all consumer groups

Offset translation: MM2 maintains mapping

- Source offset 500 → Destination offset 502 (may differ)
- Consumer migrated from source → destination sees correct position

Kafka Advanced Internals

Q26. How does Kafka handle leader election?

Difficulty: Hard

Partition leader election:

- Controller (one broker per cluster) manages elections
- Maintains list of ISR (In-Sync Replicas) per partition

When leader fails:

1. Controller detects via ZooKeeper/KRaft session expiry
2. Selects new leader from ISR (highest priority first)
3. Broadcasts new leader info to all brokers
4. Producers/consumers update their metadata → connect to new leader

If no ISR available:

- `unclean.leader.election.enable = false` (default, safe): wait for ISR
- `unclean.leader.election.enable = true`: elect any replica (risk data loss)

Preferred leader:

First replica in assignment list is "preferred"
 auto.leader.rebalance.enable = true → restore preferred leader after recovery
 Avoids hot-spot: all partitions on one broker after recovery

Q27. What are Kafka producer internals (batching, compression)?

Difficulty: Hard

Producer memory model:

RecordAccumulator: per-partition deque of ProducerBatch
 Sender thread: drains batches to brokers

Message flow:

send(record) → serialize → partition → RecordAccumulator buffer
 Sender thread: when batch full OR linger.ms elapsed → send to broker

Batching parameters:

batch.size = 16384 # bytes per batch (increase for throughput)
 linger.ms = 5 # wait 5ms before sending if batch not full
 buffer.memory = 33554432 # total memory for all batches
 max.block.ms = 60000 # how long to block if buffer full

Compression:

compression.type = snappy | lz4 | zstd | gzip | none
 Compression per batch (better ratio with larger batches)
 CPU vs network trade-off: lz4=fast, zstd=best ratio

Throughput formula:

throughput = batch_size / (message_size + linger.ms × send_rate)
 linger.ms=0 → low latency, small batches, lower throughput
 linger.ms=50 → higher latency, larger batches, higher throughput

Q28. What is Kafka consumer internals (fetch, partition assignment)?

Difficulty: Hard

Consumer fetch model:

Fetch request: consumer sends to each leader broker
 fetch.min.bytes: wait until broker has this many bytes
 fetch.max.wait.ms: max wait time
 max.partition.fetch.bytes: max per partition per request
 Multiple partitions from same broker → batched

Processing model:

poll() → fetch → process → commit → repeat

Heartbeat thread (separate from poll thread):

Sends heartbeats every heartbeat.interval.ms
 If no heartbeat for session.timeout.ms → consumer dead → rebalance
 If no poll for max.poll.interval.ms → consumer stuck → rebalance

Key constraint: processing inside poll loop must finish < max.poll.interval.ms

If slow: reduce max.poll.records or increase max.poll.interval.ms

Partition assignment (group protocol):

JoinGroup: all consumers send JoinGroup request to coordinator
 Leader elected: computes assignment based on strategy
 SyncGroup: leader sends assignment to coordinator
 Coordinator: distributes assignment to all members

Q29. What is the Kafka controller?**Difficulty: Hard**

Controller responsibilities:

- Partition leader election on broker failure
- Broker registration/deregistration
- Topic/partition creation/deletion
- Reassignment of partitions

Controller election (ZooKeeper mode):

First broker to create /controller znode wins
Others watch /controller → elect new if ephemeral node disappears

Controller election (KRaft mode):

Raft consensus among controller nodes (3 or 5)
Separate controller quorum from data brokers (optional)

Controller activities:

Keeps in-memory state: leader for each partition, ISR list
Persists to ZooKeeper/metadata log
Sends LeaderAndIsr requests to brokers after election

Performance bottleneck (old ZooKeeper):

All partition changes go through single controller
Limit: ~200K partitions per cluster

KRaft improvement:

Event-sourced metadata log
Millions of partitions supported
Faster controller failover (<1s vs 10-30s)

Q30. What are Kafka segment files and index files?**Difficulty: Hard**

Each partition's data directory:

```
00000000000000000000000000000000.log      # segment: messages
00000000000000000000000000000000.index    # offset index: offset → file position
00000000000000000000000000000000.timeindex # time index: timestamp → offset
0000000000000000000000999999.log          # next segment (starts at offset 999999)
```

Segment files:

Active segment: current write target (append-only)
log.segment.bytes = 1073741824 # 1GB default
New segment created when: size limit OR time limit (log.roll.ms)

Index file (sparse, not every offset):

[offset, position] pairs for binary search
log.index.interval.bytes = 4096 # index every 4KB
Find offset 0: binary search → nearest lower indexed offset → scan from there

Deletion:

Delete oldest segment files when retention exceeded
Or compact (keep latest per key)
Segment delete is atomic (file rename + delete)

Q31. What is Kafka quotas and throttling?**Difficulty: Medium**

Kafka quotas: limit resource usage per client/user/broker

Types:

- Producer quota: bytes/sec per producer
- Consumer quota: bytes/sec per consumer
- Request quota: % of broker thread time per client

Configuration:

```
kafka-configs.sh --add-config 'producer_byte_rate=1048576' \  
  --entity-type clients --entity-name my-producer
```

Throttling mechanism:

- Broker calculates quota violation
- Adds delay to response (throttle-time-ms in response header)
- Client backs off automatically

Use case:

- Prevent runaway consumers from overwhelming brokers
- Fair sharing between multiple producers
- Protect downstream systems from consumer bursts

Monitoring:

```
kafka.server:type=ClientQuotaManager,quota-type=fetch,user=x,client-id=y  
throttle-time metric per client
```

Q32. What are Kafka Streams state stores?

Difficulty: Hard

State stores: local storage for stateful operations (aggregations, joins)

Types:

- In-memory: fast, lost on restart (use for transient state)
- RocksDB (default): persistent, survives restart
- Custom: implement `StateStore` interface

Changelog topics:

- Every state store has a Kafka changelog topic
- Each state update → written to changelog
- On recovery: replay changelog to restore state

Interactive queries:

```
Access state stores directly from application  
streams.store(storeName, QueryableStoreTypes.keyValueStore())  
store.get(key) // point lookup  
store.range(from, to) // range scan
```

Standby replicas:

- `num.standby.replicas = 1`
- second instance pre-loads state store
- fast failover (no replay from beginning)

Global state stores:

- `GlobalKTable`: replicated to ALL instances
- enrichment lookups without network calls
- no partitioning by key

Q33. What is Kafka broker configuration for production?**Difficulty:**
Medium

```
# Replication
default.replication.factor = 3
min.insync.replicas = 2
unclean.leader.election.enable = false

# Log/Retention
log.retention.hours = 168          # 7 days
log.segment.bytes = 1073741824    # 1GB segments
log.retention.check.interval.ms = 300000 # check every 5min

# Performance
num.network.threads = 3            # network I/O threads
num.io.threads = 8                # disk I/O threads
socket.send.buffer.bytes = 102400
socket.receive.buffer.bytes = 102400
socket.request.max.bytes = 104857600 # 100MB max request

# JVM heap
# KAFKA_HEAP_OPTS="-Xmx6g -Xms6g" # 6GB for large clusters

# Monitoring
kafka.metrics.reporters = io.confluent.metrics.reporter.ConfluentMetricsReporter
```

Q34. What is Kafka transaction coordinator?**Difficulty: Hard**

Transaction coordinator: broker that manages transactions for a producer
 Assigned based on: `hash(transactional.id) % __transaction_state` partitions

Transaction lifecycle:

1. `producer.initTransactions()` → register with coordinator
2. `producer.beginTransaction()` → begin epoch
3. produce messages + `sendOffsetsToTransaction`
4. `producer.commitTransaction()` → commit marker to all partitions
 or `producer.abortTransaction()` → abort marker

Transaction log: `__transaction_state` (50 partitions default)

Stores: txn state per `transactional.id`

Zombie fencing:

Each transaction has an epoch (monotonically increasing)
 Old producer instance (zombie) rejected if `epoch < current`
 Prevents duplicate commits from crashed-then-recovered producers

`transactional.id`: unique per producer logic

Different instances of same service MUST have different `transactional.id`
 Or: use `transactional.id` + partition suffix

Q35. What is the ISR (In-Sync Replicas) and why does it matter?**Difficulty:**
Medium

ISR: set of replicas that are fully caught up to the leader

Replica falls out of ISR when:

replica.lag.time.max.ms (default 30s): too slow to replicate
 Replica is paused/crashed

acks=all behavior:

Wait for ALL ISR members to acknowledge
 If ISR = {leader, replica1}: wait for both
 If ISR = {leader} (replica fell out): only leader acks needed

min.insync.replicas:

Minimum ISR size for acks=all writes
 If ISR < min.insync.replicas → NotEnoughReplicasException
 Prevents losing data when all replicas are caught-up count drops

Example (3 brokers, min.insync.replicas=2):

Normal: ISR={0,1,2}, writes succeed
 Broker 1 down: ISR={0,2}, writes succeed (2 >= min.insync.replicas)
 Brokers 1,2 down: ISR={0}, writes fail (1 < 2)
 → protects against data loss at cost of availability

Q36. What is Kafka's message format and headers?

Difficulty:
Medium

Record format (v2, Kafka 0.11+):

baseOffset, batchSize, magic, attributes, lastOffsetDelta,
 firstTimestamp, maxTimestamp, producerId, producerEpoch, baseSequence,
 records: [{ length, attributes, timestampDelta, offsetDelta, key, value, headers }]

Record headers: key-value metadata attached to messages

Correlation ID for distributed tracing
 Schema version
 Source system identifier
 Event type / routing information

// Go producer with headers:

```
producer.Produce(&kafka.Message{
    TopicPartition: kafka.TopicPartition{Topic: "orders", Partition: kafka.PartitionAny},
    Key:    []byte(orderID),
    Value: orderJSON,
    Headers: []kafka.Header{
        {Key: "trace-id", Value: []byte(traceID)},
        {Key: "event-type", Value: []byte("order.created")},
        {Key: "schema-version", Value: []byte("v2")},
    },
}, nil)
```

Q37. What is Kafka compaction with tombstones?

Difficulty: Hard

Tombstone: message with key K and null value

→ instructs compaction to delete key K from the log

Use case: user deletes account → publish tombstone → all consumers remove from state

Timeline:

t=1: SET user:42 → {"name":"Alice"}
 t=2: SET user:42 → {"name":"Alice Updated"}
 t=3: DELETE user:42 → null (tombstone)

After compaction: only tombstone remains for user:42
 After delete.retention.ms: tombstone also deleted

Important:

Consumers must process tombstones! (delete from local store)
 Tombstones retained for delete.retention.ms (default 24h)
 → gives consumers time to see and process the delete

Compaction guarantees:

Latest value per key is always preserved
 Consumer starting from offset 0 sees all current state
 No data loss for latest values

Q38. How do you handle poison pill messages in Kafka?

Difficulty:
Medium

Poison pill: message that causes consumer to crash/hang on every retry

Problem: consumer retries indefinitely → stuck, lag grows

Strategies:

1. Dead Letter Topic (DLT):

After N retries → publish to orders.DLT → continue
 Process DLT manually or with different logic

2. Skip and log:

Catch exception → log error → commit offset → continue
 Risk: silently dropping messages

3. Retry topic + backoff:

Retry-1 topic (5s delay), Retry-2 (30s), Retry-3 (5min) → DLT
 Exponential backoff without blocking main consumer

```
// Go example (kafka-go or similar):
const maxRetries = 3
for retries := 0; retries < maxRetries; retries++ {
    err := processMessage(msg)
    if err == nil { break }
    if retries == maxRetries-1 {
        publishToDLT(msg, err) // publish to dead letter topic
    }
    time.Sleep(time.Duration(math.Pow(2, float64(retries))) * time.Second)
}
session.MarkMessage(msg, "") // always commit
```

Q39. What is Kafka's max message size configuration?

Difficulty: Easy

Default max message size: 1MB

Producer:

max.request.size = 1048576 # 1MB default

Broker:

message.max.bytes = 1048576 # 1MB default

Consumer:

```
max.partition.fetch.bytes = 1048576 # 1MB default
fetch.max.bytes = 52428800 # 50MB total response
```

Change for large messages:

```
All three must be consistent
Producer max.request.size <= Broker message.max.bytes
Consumer max.partition.fetch.bytes >= Broker message.max.bytes
```

For very large payloads (>1MB):

```
Option 1: Store in S3/DB, send reference in Kafka message
Option 2: Split into chunks with sequence numbers
Option 3: Increase all size limits (impacts memory and throughput)
```

Q40. What is Kafka's retention and disk management?

Difficulty:
Medium

Retention policies:

```
Time-based: log.retention.hours = 168 (7 days default)
Size-based: log.retention.bytes = -1 (unlimited default)
Both: whichever triggers first
```

Per-topic override:

```
kafka-configs.sh --alter --add-config retention.ms=86400000
--entity-type topics --entity-name my-topic
```

Disk calculation:

```
Total disk = partitions × replicas × (messages/sec × avg_msg_size × retention_seconds)
Example: 100K msg/sec × 1KB × 3 replicas × 7 days = ~180TB
Per broker (10 brokers): 18TB each
```

Disk monitoring:

```
broker_disk_space_used metric (JMX)
Alert at 70% disk usage → trigger cleanup or expand
```

Log deletion is segment-level (not message-level):

```
Oldest segment deleted when retention exceeded
Current segment never deleted (still being written)
→ actual retention = configured retention + up to 1 segment size
```

Q41. #### Q41-Q50: Additional Kafka Topics

Difficulty:
Medium

Q	Topic
Q41	Kafka rack awareness for cross-rack replica distribution
Q42	kafka-reassign-partitions.sh for rebalancing data
Q43	Monitoring Kafka: JMX metrics, Prometheus, Grafana
Q44	Kafka security: SSL, SASL/PLAIN, SASL/SCRAM, ACLs
Q45	Kafka timeouts: request.timeout, delivery.timeout, session.timeout
Q46	Consumer pause/resume for backpressure

Q47	Kafka message ordering guarantees (per partition, not global)
Q48	Kafka Tiered Storage (off-heap to S3 for infinite retention)
Q49	kafka-consumer-groups.sh for lag inspection and offset reset
Q50	Kafka KSQL / ksqlDB for streaming SQL on Kafka

RabbitMQ Fundamentals

Q42. What is RabbitMQ's AMQP model?

Difficulty:
Medium

AMQP model: Producer → Exchange → Queue → Consumer

Exchange types:

- Direct: route by exact routing key match
- Topic: route by pattern matching (wildcards)
- Fanout: broadcast to all bound queues
- Headers: route by message header attributes

Binding: link between exchange and queue with optional routing key

Flow:

1. Producer publishes message to Exchange with routing key "order.created"
2. Exchange routes to matching bound queues
3. Consumer receives from queue, processes, sends ACK
4. Broker removes message on ACK (or re-queues on NACK)

Key difference from Kafka:

- Messages DELETED after consumption (not retained)
- Push-based by default (broker pushes to consumer)
- Rich routing logic (exchanges, bindings)
- Dead letter exchanges for failed messages

Q43. What are RabbitMQ exchange types?

Difficulty:
Medium

Direct Exchange:

- routing key = queue binding key → exact match
- Use: point-to-point, specific routing

Exchange: order_exchange (direct)

Queue: payments_queue, bound with key "payment"

Queue: shipping_queue, bound with key "shipping"

Publish with key "payment" → only payments_queue receives

Topic Exchange:

- routing key supports wildcards: * (one word), # (zero or more words)
- Use: flexible publish-subscribe

Queue: audit_queue, bound with "order.#" → matches order.created, order.updated
 Queue: created_queue, bound with "/*.created" → matches order.created, user.created

Fanout Exchange:

Ignores routing key, delivers to ALL bound queues
 Use: broadcast notifications, event fanout

Headers Exchange:

Route by message header attributes (not routing key)
 Use: complex routing requirements, legacy systems

Q44. What are RabbitMQ message acknowledgements?

Difficulty:
Medium

ACK modes:

Auto-ACK: broker removes message immediately when delivered (risky!)
 Manual ACK: consumer explicitly ACKs after processing

Manual ACK:

```
d.Ack(false)    // ack this message
d.Ack(true)     // ack all up to and including this message (multiple)
d.Nack(false, true) // nack, requeue=true (put back to queue front)
d.Nack(false, false) // nack, requeue=false → dead letter exchange
d.Reject(true)  // reject + requeue
d.Reject(false) // reject → dead letter
```

prefetch_count (QoS):

ch.Qos(10, 0, false) // max 10 unacked messages per consumer
 Consumer receives up to 10 msgs, must ACK before getting more
 Prevents overwhelming slow consumers
 Fair dispatch: messages distributed based on consumer capacity

// Go (amqp091-go):

```
d.Ack(false)    // false = don't multiple ack
```

Q45. What is the RabbitMQ dead letter exchange (DLX)?

Difficulty:
Medium

DLX: messages routed to alternate exchange when:

- Message is nacked with requeue=false
- Message TTL expires
- Queue length limit exceeded

Configuration:

x-dead-letter-exchange: name of DLX
 x-dead-letter-routing-key: routing key for DLX (optional)

// Go (amqp091-go):

```
ch.QueueDeclare("orders.queue", true, false, false, false, amqp.Table{
    "x-dead-letter-exchange":    "orders.dlx",
    "x-dead-letter-routing-key": "orders.dead",
    "x-message-ttl":             int32(60000), // 60s TTL
})
```

// Declare DLX (fanout exchange)

```
ch.ExchangeDeclare("orders.dlx", "fanout", true, false, false, false, nil)
ch.QueueDeclare("orders.dead.queue", true, false, false, false, nil)
```

```
ch.QueueBind("orders.dead.queue", "", "orders.dlx", false, nil)
```

```
// Monitor dead letters → alert on failed processing
```

Q46. What is message durability in RabbitMQ?

Difficulty: Easy

Two components of durability:

1. Durable queue: survives broker restart
`QueueDeclare(name, durable=true, ...)`
2. Persistent message: written to disk
`Publishing.DeliveryMode = 2 // amqp.Persistent`
 Default (1) = in-memory only

BOTH required for durability:

Durable queue + transient message → message lost on restart
 Non-durable queue + persistent message → queue lost on restart

Performance impact:

Persistent messages: fsync to disk before ACK → slower
 Non-persistent: RAM only → faster but not durable
 Compromise: lazy queues (paged to disk proactively)

Quorum queues (modern approach):

Replicated across multiple nodes using Raft
 No need for durable+persistent combo
 Preferred over classic durable queues for HA
`ch.QueueDeclare("orders", true, false, false, false, amqp.Table{`
`"x-queue-type": "quorum",`
`})`

Q47. What are RabbitMQ quorum queues vs classic queues?

Difficulty: Hard

Classic Queue:

Single node (optionally mirrored)
 Mirroring: ha-mode policy (synchronous replication)
 Issues: split-brain possible, complex leadership
 Deprecated for HA use

Quorum Queue (recommended for HA):

Raft consensus across N nodes
 Majority quorum for writes ($N/2+1$)
 Leader + followers
 Strong ordering guarantees
 At-least-once delivery

Differences:

	Classic	Quorum
Durability	Configurable	Always durable
HA	Optional	Built-in (Raft)
Ordering	Yes	Yes
Priority	Yes	No
TTL	Yes	Yes
DLX	Yes	Yes
Transient	Yes	No
Lazy	Yes	Always (default)

Migration:

```
kubectrl exec rabbitmq -- rabbitmqctl set_policy quorum_queues "^" \
  '{"x-queue-type":"quorum"}' --apply-to queues
```

Q48. What is RabbitMQ channel and connection?

Difficulty: Easy

Connection: TCP connection to RabbitMQ broker

Expensive to create (~100ms, OS resources)

Should be long-lived, connection-per-service

Channel: lightweight multiplexed logical connection

Multiple channels per TCP connection

Channel-per-goroutine pattern

Operations: declare queues/exchanges, publish, consume, ACK

// Go pattern:

```
conn, _ := amqp.Dial("amqp://guest:guest@localhost:5672/")
defer conn.Close()
```

// One channel per goroutine

```
ch, _ := conn.Channel()
defer ch.Close()
```

// Don't share channels between goroutines (not goroutine-safe)

// Don't open/close channels per-message (expensive)

// Open channel once, reuse for lifetime of goroutine

Connection failure handling:

```
amqp.NotifyClose(make(chan *amqp.Error)) // watch for disconnect
Reconnect with exponential backoff
```

Q49. What is RabbitMQ prefetch and fair dispatch?

Difficulty: Medium

Without prefetch: RabbitMQ round-robins messages to consumers

Consumer A (fast) and Consumer B (slow) each get 50%

But Consumer B can't keep up → A is idle

With prefetch (QoS):

```
ch.Qos(prefetch_count, 0, false)
```

Consumer gets at most N unacked messages

Once consumer ACKs one → gets another

Fair dispatch example (prefetch=1):

Consumer A (fast): processes quickly → ACKs → gets next message

Consumer B (slow): has 1 unacked → gets no more messages until it ACKs

Result: fast consumer gets more messages = fair by capacity

prefetch_count tuning:

Too low (1): fine-grained fairness but more round-trips

Too high (1000): consumer holds many messages (memory usage, slower rebalance)

Typical: 10-50 for good balance

Global vs per-consumer:

```
ch.Qos(10, 0, false) // per-consumer prefetch (global=false)
```

```
ch.Qos(100, 0, true) // global prefetch (all consumers on channel)
```

Q50. What is the shovel and federation plugin in RabbitMQ?**Difficulty: Medium**

Shovel: moves messages between queues/exchanges

Source: queue or exchange (local or remote)

Destination: queue or exchange (local or remote)

Use cases:

- Move messages between brokers
- Bridge queue → exchange (fan-out to multiple consumers)
- Geo-replication

Federation: links exchanges/queues across brokers

Federated exchange: messages from remote exchange flow in

Federated queue: consumers pull from multiple brokers

Use cases:

- Multi-datacenter RabbitMQ
- Message routing across clusters without full replication

vs Kafka MirrorMaker:

Shovel: simple, synchronous message transfer

Federation: federated topology, more complex routing

MirrorMaker: full log replication with offset translation

Q51. What is RabbitMQ stream queue?**Difficulty: Hard**

Stream Queue (RabbitMQ 3.9+): Kafka-like append-only log within RabbitMQ

Messages retained (not deleted after consumption)

Multiple independent consumers with different offsets

Time-based and offset-based replay

Use case:

When you want replay semantics but are already invested in RabbitMQ

Differences from classic/quorum queues:

Messages NOT deleted after consumption

Consumer can re-read from any offset

Higher throughput (sequential disk I/O)

Offset tracking:

Consumer declares with x-stream-offset header

"first", "last", "next", specific offset, or timestamp

// Go (rabbitmq-stream-go client):

```
consumer, _ := env.NewConsumer("orders-stream",
    func(ctx consumerContext, msg *amqp.Message) {
        processOrder(msg)
    },
    stream.NewConsumerOptions().
        SetOffset(stream.OffsetSpecification{}.First()),
)
```

Q52. What is RabbitMQ management and monitoring?**Difficulty: Easy**

Management Plugin:

HTTP API: `http://localhost:15672/api/`
Web UI: `queues, exchanges, connections, messages, consumers`

Key metrics:

`messages_ready`: messages waiting for consumers
`messages_unacknowledged`: delivered but not yet ACKed
`message_stats.publish_rate`: messages/sec
`message_stats.deliver_rate`: messages/sec to consumers
`consumer_utilisation`: % time consumer is receiving messages

Alert on:

`messages_ready > threshold` → consumers can't keep up
`messages_unacknowledged` growing → slow consumers / stuck processing
`consumer_utilisation < 50%` → consumers waiting (need more consumers)

Prometheus plugin:

`rabbitmq_queue_messages{queue="orders"}` gauge
`rabbitmq_queue_messages_unacked{queue="orders"}`

Health check:

`GET /api/healthchecks/node` → `{"status":"ok"}`
`rabbitmq-diagnostics check_running`

Q53. What is RabbitMQ high availability (HA) setup?

Difficulty: Hard

Cluster: multiple RabbitMQ nodes sharing users/vhosts/exchanges
Queue distribution: queues on one node by default

Quorum Queues (recommended):

Raft replication across N nodes
Minimum 3 nodes for quorum (majority = 2)
Automatic leader failover

`rabbitmq-plugins enable rabbitmq_peer_discovery_k8s`
Kubernetes: auto-join on startup

Classic Mirrored Queues (deprecated):

`ha-mode`: `all/exactly/nodes` policy
Synchronous replication to mirrors
Issues: split-brain, complex recovery

Load balancing:

HAProxy/Nginx in front of RabbitMQ cluster
Multiple nodes → clients connect to any
Queue leader is the canonical node for that queue

Network partition handling:

`cluster_partition_handling = pause_minority` (safe)
→ minority partition pauses (refuses connections)
→ prevents split-brain data divergence
`cluster_partition_handling = autoheal` (risky)
→ larger partition continues, smaller merges on rejoin

Q54. What is the RabbitMQ vs Kafka decision matrix?

Difficulty: Easy

Choose RabbitMQ when:

- ✓ Complex routing logic (topic/header/fanout exchanges)
- ✓ Request-reply pattern (RPC over AMQP)
- ✓ Task queues with priority
- ✓ Per-message TTL
- ✓ Message acknowledgement per message
- ✓ Small team, simpler operations
- ✓ Existing AMQP ecosystem

Choose Kafka when:

- ✓ High throughput (100K+ msg/sec)
- ✓ Message replay (event sourcing, audit)
- ✓ Multiple independent consumer groups
- ✓ Stream processing (Kafka Streams, Flink)
- ✓ Long retention (days/weeks/forever)
- ✓ CDC (Change Data Capture)
- ✓ Ordered processing per entity

Both are wrong, consider:

SQS: fully managed, simple, AWS-native, lower volume

Redis Streams: lower infra, embedded in Redis stack

NATS: lightweight, cloud-native, lower operational overhead

Q55. What is RabbitMQ lazy queues?

Difficulty:
Medium

Lazy queue: messages paged to disk immediately instead of memory

Classic behaviour: messages in RAM, spill to disk when memory pressure

Lazy: immediately persisted to disk → more predictable memory use

Benefits:

Handles large backlogs without OOM

More predictable broker memory usage

Good for: bursty producers, slow consumers

Cost:

Slightly higher latency (disk write on publish)

Higher I/O on publish

Enable:

Policy: `rabbitmqctl set_policy lazy "^" '{"queue-mode":"lazy"}' --apply-to queues`

Or per-queue:

`x-queue-mode: lazy` (in `QueueDeclare` args)

Note: Quorum queues are always lazy by default

Classic queues: choose lazy explicitly for stability

Q56. What is message priority in RabbitMQ?

Difficulty:
Medium

Priority queue: messages with higher priority delivered first

Declare priority queue:

`x-max-priority: 10` (support 1-10 priority levels)

// Go:

```
ch.QueueDeclare("orders.priority", true, false, false, false, amqp.Table{
```

```

    "x-max-priority": int32(10),
})

// Publish with priority:
ch.Publish("", "orders.priority", false, false, amqp.Publishing{
    Priority: 9, // high priority
    Body: body,
})

Scheduling:
    RabbitMQ delivers highest priority messages first
    Same priority: FIFO

Use cases:
    Premium user orders before standard
    Critical alerts before informational

Kafka does NOT support priority (all messages equal)
→ for priority: use multiple topics or RabbitMQ

```

Q57. What are RabbitMQ virtual hosts (vhosts)?

Difficulty: Easy

vhost: logical namespace separating exchanges, queues, bindings
 Default vhost: /
 Isolation: queues/exchanges in different vhosts are completely separate

Use cases:
 Multi-tenancy: one vhost per tenant
 Environment separation: staging vs prod on same cluster
 Application isolation: service A's queues separate from service B

```

// Create vhost:
rabbitmqctl add_vhost myapp
rabbitmqctl set_permissions -p myapp myuser ".*" ".*" ".*"

// Go: specify vhost in connection string
amqp.Dial("amqp://user:pass@host:5672/myapp")
// or
amqp.Dial("amqp://user:pass@host:5672/%2Fmyvhost") // URL encode "/"

Vhost limitations:
    No cross-vhost message routing (different from Kafka topics)
    Each vhost has its own resource usage
    vhost = unit of isolation, not unit of performance

```

Q58. What is RabbitMQ consumer concurrency?

Difficulty:
Medium

Options for concurrent consumers in Go:

```

// Option 1: Multiple goroutines sharing one channel (wrong!)
// Channels are NOT goroutine-safe in amqp091-go

// Option 2: One channel per goroutine (correct)
func startWorkers(conn *amqp.Connection, n int, queue string) {
    for i := 0; i < n; i++ {
        go func() {

```

```

        ch, _ := conn.Channel()
        ch.Qos(10, 0, false)
        msgs, _ := ch.Consume(queue, "", false, false, false, false, nil)
        for d := range msgs {
            processMessage(d.Body)
            d.Ack(false)
        }
    }()
}
}

// Option 3: Multiple consumer tags on same channel (AMQP allows this, avoid in Go)

// Prefetch per goroutine:
// 10 goroutines × prefetch=10 = 100 in-flight messages per connection

// Connection pooling for multiple goroutines:
// 1 connection, N channels (goroutines) = efficient

```

Q59. What is the AMQP heartbeat?

Difficulty: Easy

Heartbeat: periodic exchange between client and broker
 Detects dead connections (TCP can stay "connected" while peers are dead)

Configuration:

```

amqp.DialConfig("amqp://...", amqp.Config{
    Heartbeat: 10 * time.Second, // send heartbeat every 10s
})

```

If no heartbeat received in $2 \times$ heartbeat interval:
 Connection considered dead → close + reconnect

Default: 60 seconds (heartbeat every 60s, timeout at 120s)
 Recommended production: 10 seconds

Common issue: firewall/NAT timeout closes idle TCP connections
 Heartbeat prevents this (keeps TCP alive)
 Set lower than firewall idle timeout

vs Kafka:

Kafka uses its own heartbeat protocol within consumer group
 Separate from TCP keepalive

Q60. What is a RabbitMQ policy?

Difficulty: Medium

Policy: dynamically apply settings to queues/exchanges by name pattern
 No code change needed
 Overrides queue/exchange arguments

```

// rabbitmqctl
rabbitmqctl set_policy ha-all "^orders\." \
    '{"ha-mode":"all","ha-sync-mode":"automatic"}' \
    --apply-to queues

rabbitmqctl set_policy dlx "^" \
    '{"dead-letter-exchange":"dlx","dead-letter-routing-key":"dead"}' \

```

```
--apply-to queues

rabbitmqctl set_policy ttl "^temp\\.\" \
  '{"message-ttl':60000}' \
  --apply-to queues

rabbitmqctl list_policies

// Policy parameters (all can be set):
// ha-mode, ha-params, ha-sync-mode
// dead-letter-exchange, dead-letter-routing-key
// message-ttl, expires
// max-length, max-length-bytes
// overflow (drop-head or reject-publish)
// queue-mode (lazy)
```

Q61. What is the difference between RabbitMQ nack+requeue and nack+DLX?

Difficulty:
Medium

```
nack(requeue=true):
  Message goes BACK to queue immediately
  Risk: infinite retry loop → CPU thrashing
  Use: transient failures (DB temporarily unavailable)
  Add: max retries via x-death header count

nack(requeue=false):
  If queue has DLX configured: message → DLX → dead letter queue
  If no DLX: message discarded
  Use: processing errors, schema errors, business logic errors

x-death header:
  RabbitMQ adds to message each time it enters DLX
  Contains: queue name, reason, count, time
  Use to limit retries:
    if death_count >= 3: send to permanent DLQ, alert humans

// Go: check x-death count
deaths, _ := d.Headers["x-death"].([]interface{})
deathCount := 0
if len(deaths) > 0 {
  death := deaths[0].(amqp.Table)
  deathCount = int(death["count"].(int64))
}
if deathCount >= 3 {
  alertOpsTeam(d); d.Nack(false, false); return
}
```

Messaging Patterns & Design

Q62. What is the competing consumers pattern?

Difficulty: Easy

Multiple consumers reading from the same queue, competing for messages
→ horizontal scaling of message processing

Kafka: partitions=3, consumer group=3 → 1 partition each
 RabbitMQ: queue=1, multiple consumers → each gets different messages

When to use:

Processing is stateless (any consumer can handle any message)
 Need to scale throughput by adding workers
 Order of processing doesn't matter across consumers

With ordering:

Kafka: route by key → same partition → same consumer → ordered
 RabbitMQ: no ordering guarantee across competing consumers

```
// Go (RabbitMQ): spawn N goroutines consuming same queue
for i := 0; i < numWorkers; i++ {
    go startConsumer(conn, "orders.queue")
}
// Each goroutine gets different messages (RabbitMQ distributes)
```

Q63. What is the fan-out (publish-subscribe) pattern?

Difficulty: Easy

One producer → one message → multiple consumers each get a copy

Kafka:

Same topic → multiple consumer groups → each group reads independently
 Group A (email service), Group B (analytics), Group C (audit)
 All three get every order event

RabbitMQ:

Fanout exchange → bind multiple queues → each gets copy
 Or: Topic exchange with # binding per queue

```
ExchangeDeclare("notifications", "fanout", true, ...)
QueueBind("email.queue", "", "notifications", ...) // receives all
QueueBind("sms.queue", "", "notifications", ...) // receives all
QueueBind("push.queue", "", "notifications", ...) // receives all
```

Kafka advantage: consumer groups added later without producer change
 RabbitMQ: add new queue binding without producer change
 Both support extensibility well

Q64. What is the request-reply (RPC) pattern over messaging?

Difficulty:
Medium

Pattern: caller sends request + correlation ID, waits for reply

RabbitMQ (natural fit):

1. Caller: publish to request queue with reply-to=temp-queue, correlation-id=uuid
2. Server: consume request, process, publish to reply-to queue
3. Caller: consume from temp queue, match by correlation-id

```
// Go (amqp):
corrID := uuid.New().String()
replyQueue, _ := ch.QueueDeclare("", false, true, true, false, nil) // temp queue

ch.Publish("", "rpc.queue", false, false, amqp.Publishing{
    ReplyTo:      replyQueue.Name,
    CorrelationId: corrID,
```

```

    Body:      request,
  })

  msgs, _ := ch.Consume(replyQueue.Name, "", true, false, false, false, nil)
  for d := range msgs {
    if d.CorrelationId == corrID {
      // got response
      break
    }
  }
}

```

Kafka: not ideal for RPC (pull-based, high latency)
 Use gRPC or HTTP for synchronous request-reply
 Kafka: for async fire-and-forget

Q65. What is the saga pattern with messaging?

Difficulty: Hard

Saga: distributed transaction via sequence of local transactions + events

Choreography (event-driven):

```

OrderService → OrderCreated event
PaymentService → listens, charges card → PaymentCharged event
InventoryService → listens, reserves stock → StockReserved event
ShippingService → listens, creates shipment

```

Failure: StockOutOfStock event

```

PaymentService listens → RefundPayment compensation
OrderService listens → CancelOrder compensation

```

Kafka topics:

```

orders (OrderCreated, OrderCancelled)
payments (PaymentCharged, PaymentRefunded)
inventory (StockReserved, StockReleased)

```

Orchestration (centralized):

```

SagaOrchestrator → step1: charge payment
→ success: step2: reserve stock
→ success: step3: create shipment
→ failure at any step: run compensations

```

Tools: Temporal (Go SDK), AWS Step Functions, Conductor

Temporal is the production standard for sagas in Go:

```

workflow.ExecuteActivity(ctx, chargePayment, order)
workflow.ExecuteActivity(ctx, reserveStock, order)

```

Q66. What is the outbox pattern with Kafka?

Difficulty: Hard

Outbox: atomic write to DB + eventual publish to Kafka

Solves: "write to DB and publish to Kafka" dual-write problem

Step 1: Write atomically

```

BEGIN;
INSERT INTO orders VALUES (...);
INSERT INTO outbox (event_type, payload, created_at) VALUES (...);
COMMIT;

```

Step 2: Publish from outbox

Option A: Polling (simple)

```
SELECT * FROM outbox WHERE published=false LIMIT 100 FOR UPDATE SKIP LOCKED
kafka.Produce(each event)
UPDATE outbox SET published=true WHERE id=...
```

Option B: Debezium CDC (production)

```
PostgreSQL WAL → Debezium → Kafka
Debezium reads outbox table changes → publishes to Kafka
Near-real-time, no polling needed
```

Guarantees:

```
At-least-once delivery to Kafka (idempotent consumer required)
No phantom events (only committed transactions publish)
No message loss (DB durability before Kafka publish)
```

Q67. What is consumer idempotency and how do you implement it?**Difficulty:
Medium**

Idempotency: processing same message twice = same result as once
 Required for: at-least-once delivery, Kafka EOS, RabbitMQ retries

Implementation strategies:**1. Natural idempotency:**

```
"SET order.status = SHIPPED" → same result if run twice
```

2. Deduplication key in DB:

```
Before processing: INSERT INTO processed_msgs (msg_id) ON CONFLICT DO NOTHING
If insert succeeds → process
If conflict → already processed → skip
```

3. Redis deduplication:

```
SET processed:{msg_id} 1 EX 86400 NX
If SET succeeds → process; if fails → duplicate → skip
```

4. Conditional update:

```
UPDATE orders SET status='shipped'
WHERE order_id=? AND status='confirmed' -- only if in expected state
Rows affected = 0 → already processed (idempotent)
```

```
// Go:
```

```
func processMessage(ctx context.Context, msgID string, payload []byte) error {
    ok, _ := redis.SetNX(ctx, "processed:"+msgID, 1, 24*time.Hour).Result()
    if !ok { return nil } // already processed
    return doWork(payload)
}
```

Q68. What is backpressure in messaging systems?**Difficulty:
Medium**

Backpressure: slow consumers signal producers to slow down

Kafka:

```
Producer blocks when buffer.memory full (max.block.ms)
Consumer lag metric → alert → scale consumers or reduce producers
KEDA (Kubernetes Event-Driven Autoscaler) scales based on lag
```

RabbitMQ:

Queue depth grows → memory threshold → trigger flow control
 Publisher flow control: broker sends block/unblock to connection
 prefetch_count: limits in-flight messages per consumer

Application-level backpressure (Go):

Bounded channel as buffer:

```
jobs := make(chan []byte, 1000) // bounded = backpressure
```

producer:

```
jobs <- message // blocks if full = backpressure to sender
```

consumer:

```
for msg := range jobs { process(msg) }
```

Design principle:

Every queue / channel should have a maximum size

On overflow: drop (logs ok), block (critical), or fail-fast (API)

Never unbounded queues in production (memory will fill)

Q69. What is message ordering and how do you guarantee it?

Difficulty: Hard

Kafka ordering:

Guaranteed within a partition, NOT across partitions

Route by key → same key → same partition → ordered

key = order_id: all events for order 42 → partition-X → processed in order

key = user_id: all events for user 99 → partition-Y → ordered per user

Reprocessing: replay from beginning → same order (deterministic)

RabbitMQ ordering:

Queue: messages delivered in FIFO order

BUT: with multiple consumers → each gets different messages

Consumer A processes msg1, Consumer B processes msg2

If A is slow: B's msg2 completes before A's msg1

→ out of order from system perspective

Single consumer per queue → strict ordering

Multiple consumers → no ordering guarantee

Pattern for ordered processing with multiple consumers:

Partition by entity: hash(entity_id) → route to specific queue

N queues × 1 consumer = N parallel ordered streams

Similar to Kafka's partition model

Q70. What is the event-driven architecture and how do Kafka/RMQ fit?

Difficulty:
Medium

EDA components:

Event producers: emit events (fact, something happened)

Event channel: Kafka topic or RabbitMQ exchange

Event consumers: react to events

Events vs Commands:

Event: OrderCreated (past tense, fact, broadcast, multiple receivers)

Command: CreateOrder (imperative, one receiver, expects action)

Kafka for EDA:

Events stored durably → any new service can replay history
 Multiple consumer groups → each service independently processes
 Time-ordering within partition → causal consistency

RabbitMQ for EDA:

Better for: routing complexity (specific events to specific services)
 Fanout for broadcast, Topic for selective subscription
 No replay → events gone after consumption

Event-Carried State Transfer:

Include full entity in event (not just ID)
 Consumer doesn't need to call back producer for data
 Reduces inter-service coupling

Event Sourcing:

All state changes = events stored in Kafka
 Current state = replay of all events
 Kafka's long retention enables this

Q71. What is the dead letter queue (DLQ) pattern?**Difficulty: Easy**

DLQ: queue for messages that failed processing N times

Why:

Without DLQ: failed message retries forever (blocks queue / CPU thrash)
 With DLQ: isolate bad messages → fix bug → replay manually

Kafka DLQ (manual, no built-in):

Consumer catches error after N retries → produce to orders.DLT topic
 Monitor DLT depth → alert

```
// Go Kafka DLQ:
const maxRetries = 3
for attempt := 0; attempt < maxRetries; attempt++ {
    if err := process(msg); err == nil { break }
    if attempt == maxRetries-1 {
        publishToDLT(dltProducer, msg, err) // send to dead letter topic
    }
    time.Sleep(retryDelay(attempt))
}
```

RabbitMQ DLQ:

x-dead-letter-exchange on queue → auto-route to DLX on nack
 x-message-ttl → auto-route when TTL expires

Replay from DLQ:

Once bug is fixed: re-publish DLQ messages to original queue
 kafka-consumer-groups.sh --reset-offsets (for Kafka)

Q72. What is the transactional outbox vs change data capture?**Difficulty: Hard**

Both solve: write to DB + publish to Kafka atomically

Outbox Pattern (polling):

App writes to outbox table in same transaction

Poller: reads unpublished outbox entries → publishes to Kafka

Pros: simple, works with any DB

Cons: polling delay (seconds), DB load from polling, ordering complexity

CDC (Change Data Capture) with Debezium:

Read database WAL (transaction log) directly

Debezium publishes every row change to Kafka

Near-real-time (milliseconds from commit to Kafka)

Pros: real-time, no polling, no extra DB load, ordering preserved

Cons: more infrastructure (Kafka Connect, Debezium), WAL slot management

Hybrid (production standard):

App writes to outbox table (atomic with business data)

Debezium reads outbox table changes from WAL → publishes to Kafka

Best of both: atomic writes + near-real-time delivery

Libraries: debezium-embedded, Eventuate Tram

Q73. What is message schema evolution?

Difficulty:
Medium

Problem: as services evolve, message schema changes

Old consumers must still work with new messages (forward compatibility)

New consumers must still work with old messages (backward compatibility)

Strategies:

1. JSON: flexible but no enforced schema

Add fields: old consumers ignore unknown fields (backward compat)

Remove fields: new consumers handle missing fields

Risk: no validation, typos, implicit schema

2. Avro with Schema Registry:

Schema registered, evolution rules enforced

BACKWARD: new schema can read old data (add optional fields)

FORWARD: old schema can read new data (remove optional fields)

FULL: both

3. Protobuf: field numbers stable, add new = backward compat

Don't reuse field numbers

Use optional/repeated for new fields

4. Versioned message types:

message.type = "order.v1" vs "order.v2"

Multiple consumers for different versions (migration period)

Best practice:

Never remove required fields

Never change field types

Add new fields as optional with defaults

Version your messages (x-schema-version header or type field)

Q74. What is the event sourcing pattern with Kafka?

Difficulty: Hard

Event sourcing: all state = sequence of events stored in Kafka

Current state = replay events from offset 0

Domain events stored in Kafka topic:

order-events: [OrderCreated, ItemAdded, PaymentReceived, OrderShipped, ...]

Deriving state:

Rebuild order by replaying all events for order_id
 State = fold(events, initialState, applyEvent)

Snapshots:

Periodically snapshot current state → store in DB
 On query: load last snapshot + replay events since snapshot

Advantages:

Full audit history (every change recorded)
 Rebuild any past state (time travel)
 Multiple views from same events (CQRS read models)
 Easy event replay for new consumer services

Challenges:

Schema evolution of old events
 Event ordering (use sequence numbers)
 Eventual consistency of read models
 Growing event log (compaction for current state topics)

Kafka compacted topic = event sourcing for current state

Kafka retention + compaction = full history + current state

Q75. What is CQRS with Kafka?

Difficulty: Hard

CQRS: Command Query Responsibility Segregation

Write model: handles commands, emits events

Read model: consumes events, builds query-optimized views

Flow:

1. Client sends command: CreateOrder
2. Command handler writes to write DB + publishes OrderCreated to Kafka
3. Projection consumers:
 - a. Build OrderSummary view in Redis/Elasticsearch
 - b. Update analytics DB
 - c. Send email notification
4. Client queries read model (Redis/Elasticsearch) for fast reads

Implementation:

topics: order-commands → order-events → projections

Go Kafka consumer (projection):

```
for msg := range orderEvents {
    event := deserialize(msg)
    switch event.Type {
    case "OrderCreated":
        redis.Set(event.OrderID, buildSummary(event))
    case "OrderShipped":
        redis.HSet(event.OrderID, "status", "shipped")
    }
    commitOffset(msg)
}
```

Benefits:

- Read model tailored per query (no N+1, no complex JOINS)
- Scales reads independently from writes
- Multiple read models from same events

Q76. What is the strangler fig pattern with messaging?**Difficulty:**
Medium

Strangler Fig: incrementally replace monolith using message broker as mediator

Step 1: Monolith publishes events to Kafka (anti-corruption layer)

- MonolithService → OrderCreated event → Kafka
- New microservice consumes events → builds its own data model

Step 2: New service shadows monolith

- Requests go to BOTH monolith and new service
- Compare outputs, fix discrepancies

Step 3: Gradually shift traffic

- Feature flag: 5% → 50% → 100% to new service

Step 4: Monolith stops handling that feature

- New service now canonical
- Events still published to Kafka (downstream consumers unaffected)

Kafka advantages for strangler fig:

- Event stream = integration contract between old and new
- New services can replay history from day of launch
- No synchronous coupling between monolith and new services

Timeline: months to years for large monoliths

Risk: low (gradual, parallel running)

Go Integration

Q77. How do you use franz-go for Kafka in Go?**Difficulty:**
Medium

```
import "github.com/twmb/franz-go/pkg/kgo"

// Producer
client, err := kgo.NewClient(
    kgo.SeedBrokers("kafka:9092"),
    kgo.ProducerBatchMaxBytes(1<<20),
    kgo.RecordPartitioner(kgo.StickyKeyPartitioner(nil)),
)
defer client.Close()

record := &kgo.Record{
    Topic: "orders",
    Key:   []byte("order-42"),
    Value: orderJSON,
    Headers: []kgo.RecordHeader{
        {Key: "trace-id", Value: []byte(traceID)},
    },
}
```

```

    },
}

ctx := context.Background()
if err := client.ProduceSync(ctx, record).FirstErr(); err != nil {
    return fmt.Errorf("produce: %w", err)
}

// Async produce with callback
client.Produce(ctx, record, func(r *kgo.Record, err error) {
    if err != nil { log.Printf("produce error: %v", err) }
}))

```

Q78. How do you build a Kafka consumer in Go with franz-go?

Difficulty:
Medium

```

client, _ := kgo.NewClient(
    kgo.SeedBrokers("kafka:9092"),
    kgo.ConsumerGroup("my-service"),
    kgo.ConsumeTopics("orders"),
    kgo.WithHooks(&kgo.BasicLogger{}),
)
defer client.Close()

ctx := context.Background()
for {
    fetches := client.PollFetches(ctx)
    if fetches.IsClientClosed() { break }

    fetches.EachError(func(t string, p int32, err error) {
        log.Printf("fetch error topic=%s partition=%d: %v", t, p, err)
    })

    fetches.EachRecord(func(r *kgo.Record) {
        if err := processOrder(r.Value); err != nil {
            log.Printf("process error offset=%d: %v", r.Offset, err)
            // publish to DLT or handle
            return
        }
    })

    // Commit after processing the entire poll batch
    if err := client.CommitUncommittedOffsets(ctx); err != nil {
        log.Printf("commit error: %v", err)
    }
}

```

Q79. How do you use segmentio/kafka-go?

Difficulty:
Medium

```

import "github.com/segmentio/kafka-go"

// Writer (producer)
w := kafka.NewWriter(kafka.WriterConfig{
    Brokers: []string{"kafka:9092"},
    Topic:   "orders",
    Balancer: &kafka.Hash{}, // route by key
})

```

```

})
defer w.Close()

err := w.WriteMessages(ctx,
    kafka.Message{
        Key:    []byte("order-42"),
        Value:  orderJSON,
        Headers: []kafka.Header{
            {Key: "trace-id", Value: []byte(traceID)},
        },
    },
)

// Reader (consumer)
r := kafka.NewReader(kafka.ReaderConfig{
    Brokers: []string{"kafka:9092"},
    GroupID: "my-service",
    Topic:   "orders",
    MinBytes: 10e3, // 10KB
    MaxBytes: 10e6, // 10MB
})
defer r.Close()

for {
    m, err := r.FetchMessage(ctx)
    if err != nil { break }
    if err := processOrder(m.Value); err != nil {
        // handle error
        continue
    }
    r.CommitMessages(ctx, m) // commit only after successful processing
}

```

Q80. How do you handle Kafka consumer errors and retries in Go?

Difficulty: Hard

```

type RetryConfig struct {
    MaxAttempts int
    BaseDelay   time.Duration
}

func processWithRetry(msg kgo.Record, cfg RetryConfig) error {
    var lastErr error
    for attempt := 0; attempt < cfg.MaxAttempts; attempt++ {
        if attempt > 0 {
            delay := cfg.BaseDelay * (1 << (attempt - 1)) // exponential
            time.Sleep(delay + time.Duration(rand.Intn(100))*time.Millisecond)
        }
        if err := processMessage(msg.Value); err != nil {
            lastErr = err
            log.Printf("attempt %d failed: %v", attempt+1, err)
            if !isRetryable(err) { break }
            continue
        }
        return nil
    }
    // Send to DLT
    return publishToDLT(msg, lastErr)
}

```

```

}

func isRetryable(err error) bool {
    // Network errors: retry
    // Business logic errors: don't retry
    var netErr *net.Error
    return errors.As(err, &netErr)
}

func publishToDLT(msg kgo.Record, originalErr error) error {
    dltRecord := &kgo.Record{
        Topic: msg.Topic + ".DLT",
        Key:   msg.Key,
        Value: msg.Value,
        Headers: append(msg.Headers, kgo.RecordHeader{
            Key:   "original-error",
            Value: []byte(originalErr.Error()),
        }),
    }
    return dltClient.ProduceSync(context.Background(), dltRecord).FirstErr()
}

```

Q81. How do you build a transactional Kafka producer in Go?

Difficulty: Hard

```

// Exactly-once: consume → process → produce (all in one transaction)
client, _ := kgo.NewClient(
    kgo.SeedBrokers("kafka:9092"),
    kgo.TransactionalID("my-service-0"), // unique per producer instance
    kgo.RequiredAcks(kgo.AllISRacks()),
    kgo.ProducerBatchMaxBytes(1 << 20),
)
defer client.Close()

// Consumer (read_committed)
consumer, _ := kgo.NewClient(
    kgo.SeedBrokers("kafka:9092"),
    kgo.ConsumerGroup("my-group"),
    kgo.ConsumeTopics("input-topic"),
    kgo.FetchIsolationLevel(kgo.ReadCommitted()),
)

for {
    fetches := consumer.PollFetches(ctx)

    // Start transaction
    if err := client.BeginTransaction(); err != nil { handle(err) }

    fetches.EachRecord(func(r *kgo.Record) {
        output := transform(r.Value)
        client.Produce(ctx, &kgo.Record{Topic: "output-topic", Value: output}, nil)
    })

    // Commit offsets within transaction
    offsets := consumer.UncommittedOffsets()
    if err := client.EndTransaction(ctx, kgo.TransactionEndCommit); err != nil {
        client.EndTransaction(ctx, kgo.TransactionEndAbort)
        // consumer will reprocess (exactly-once guarantee)
    }
}

```



```
    }
}
```

Q82. How do you implement RabbitMQ consumer in Go?

Difficulty: Easy

```
import amqp "github.com/rabbitmq/amqp091-go"

func startConsumer(conn *amqp.Connection, queueName string) error {
    ch, err := conn.Channel()
    if err != nil { return fmt.Errorf("channel: %w", err) }
    defer ch.Close()

    // Fair dispatch: max 10 unacked messages at a time
    ch.Qos(10, 0, false)

    q, err := ch.QueueDeclare(queueName, true, false, false, false, nil)
    if err != nil { return fmt.Errorf("declare: %w", err) }

    msgs, err := ch.Consume(q.Name, "", false, false, false, false, nil)
    if err != nil { return fmt.Errorf("consume: %w", err) }

    for d := range msgs {
        if err := processMessage(d.Body); err != nil {
            log.Printf("process error: %v", err)
            d.Nack(false, false) // nack → DLX
            continue
        }
        d.Ack(false)
    }
    return nil
}
```

Q83. How do you implement RabbitMQ producer in Go?

Difficulty: Easy

```
func publishOrder(ch *amqp.Channel, order *Order) error {
    body, err := json.Marshal(order)
    if err != nil { return err }

    return ch.Publish(
        "orders",          // exchange
        "order.created",   // routing key
        false,             // mandatory (return if no route)
        false,            // immediate (deprecated)
        amqp.Publishing{
            ContentType: "application/json",
            DeliveryMode: amqp.Persistent, // 2 = durable
            Body:        body,
            MessageId:   uuid.New().String(),
            Timestamp:   time.Now(),
            Headers: amqp.Table{
                "trace-id":    traceID,
                "schema-version": "v1",
            },
        },
    )
}
```

```
// Channel is NOT goroutine-safe: one per goroutine
// Connection IS safe to share: one per application
```

Q84. How do you implement RabbitMQ reconnection in Go?

Difficulty: Hard

```
type RabbitMQClient struct {
    dsn      string
    conn     *amqp.Connection
    channel  *amqp.Channel
    mu       sync.Mutex
}

func (c *RabbitMQClient) connect(ctx context.Context) error {
    backoff := time.Second
    for {
        conn, err := amqp.Dial(c.dsn)
        if err == nil {
            ch, err := conn.Channel()
            if err == nil {
                c.mu.Lock()
                c.conn, c.channel = conn, ch
                c.mu.Unlock()
                go c.watchConnection(conn)
                return nil
            }
            conn.Close()
        }
        select {
        case <-ctx.Done(): return ctx.Err()
        case <-time.After(backoff):
            backoff = min(backoff*2, 30*time.Second)
        }
    }
}

func (c *RabbitMQClient) watchConnection(conn *amqp.Connection) {
    notifyClose := make(chan *amqp.Error, 1)
    conn.NotifyClose(notifyClose)
    if err := <-notifyClose; err != nil {
        log.Printf("RabbitMQ connection closed: %v, reconnecting...", err)
        c.connect(context.Background())
    }
}
```

Q85. How do you implement the outbox pattern in Go?

Difficulty: Hard

```
// PostgreSQL + Kafka outbox in Go

type OutboxPoller struct {
    db          *sql.DB
    producer    *kgo.Client
    interval    time.Duration
}

func (p *OutboxPoller) Run(ctx context.Context) {
    ticker := time.NewTicker(p.interval)
```

```

    defer ticker.Stop()
    for {
        select {
            case <-ctx.Done(): return
            case <-ticker.C: p.poll(ctx)
        }
    }
}

func (p *OutboxPoller) poll(ctx context.Context) {
    rows, err := p.db.QueryContext(ctx, `
        SELECT id, event_type, payload FROM outbox
        WHERE published_at IS NULL
        ORDER BY id LIMIT 100
        FOR UPDATE SKIP LOCKED`)
    if err != nil { log.Printf("outbox query: %v", err); return }
    defer rows.Close()

    var ids []int64
    var records []*kgo.Record
    for rows.Next() {
        var id int64; var eventType string; var payload []byte
        rows.Scan(&id, &eventType, &payload)
        ids = append(ids, id)
        records = append(records, &kgo.Record{
            Topic: eventType, Value: payload,
        })
    }

    if len(records) == 0 { return }
    if err := p.producer.ProduceSync(ctx, records...).FirstErr(); err != nil {
        log.Printf("produce: %v", err); return
    }
    p.db.ExecContext(ctx,
        "UPDATE outbox SET published_at=NOW() WHERE id=ANY($1)",
        pq.Array(ids))
}

```

Q86. How do you implement consumer lag monitoring in Go?

Difficulty:
Medium

```

import "github.com/twmb/franz-go/pkg/kadm"

func checkConsumerLag(ctx context.Context, brokers []string, group, topic string) (map[int32]int64, error) {
    admin, err := kadm.NewClient(
        kgo.SeedBrokers(brokers...),
    )
    if err != nil { return nil, err }
    defer admin.Close()

    // Get latest offsets for topic
    offsets, err := admin.ListEndOffsets(ctx, topic)
    if err != nil { return nil, err }

    // Get committed offsets for consumer group
    committed, err := admin.FetchOffsets(ctx, group)
    if err != nil { return nil, err }
}

```

```

    lag := make(map[int32]int64)
    offsets.Each(func(o kadm.ListedOffset) {
        if o.Err != nil { return }
        partition := o.Partition
        latest := o.Offset
        committed := committed.Lookup(topic, partition)
        if committed.At >= 0 {
            lag[partition] = latest - committed.At
        } else {
            lag[partition] = latest // never consumed
        }
    })
    return lag, nil
}

// Export to Prometheus:
lagGauge.With(prometheus.Labels{"partition": strconv.Itoa(int(p))}).Set(float64(l))

```

Q87. How do you test Kafka consumers in Go?

Difficulty:
Medium

```

// Option 1: testcontainers-go with real Kafka
import "github.com/testcontainers/testcontainers-go/modules/kafka"

func TestOrderConsumer(t *testing.T) {
    ctx := context.Background()

    container, err := kafka.RunContainer(ctx,
        kafka.WithClusterID("test-cluster"),
        testcontainers.WithImage("confluentinc/cp-kafka:7.5.0"),
    )
    require.NoError(t, err)
    defer container.Terminate(ctx)

    brokers, _ := container.Brokers(ctx)

    // Produce test message
    producer, _ := kgo.NewClient(kgo.SeedBrokers(brokers...))
    producer.ProduceSync(ctx, &kgo.Record{Topic: "orders", Value: orderJSON})

    // Run consumer
    consumer := NewOrderConsumer(brokers)
    processed := make(chan *Order, 1)
    go consumer.Run(ctx, func(o *Order) { processed <- o })

    select {
    case order := <-processed:
        assert.Equal(t, expectedOrder, order)
    case <-time.After(10 * time.Second):
        t.Fatal("timeout waiting for order")
    }
}

// Option 2: mock interface
type MessageProducer interface {
    Publish(ctx context.Context, topic string, key, value []byte) error
}

```

```
// Use mock in unit tests, real Kafka in integration tests
```

Q88. How do you implement a Kafka producer with circuit breaker in Go?

Difficulty: Hard

```
import "github.com/sony/gobreaker"

type SafeKafkaProducer struct {
    client  *kgo.Client
    breaker *gobreaker.CircuitBreaker
}

func NewSafeProducer(brokers []string) *SafeKafkaProducer {
    cb := gobreaker.NewCircuitBreaker(gobreaker.Settings{
        Name:      "kafka-producer",
        MaxRequests: 1,           // half-open: try 1 request
        Interval:   time.Minute, // reset counts every minute
        Timeout:    30 * time.Second, // open → half-open after 30s
        ReadyToTrip: func(counts gobreaker.Counts) bool {
            return counts.ConsecutiveFailures > 5
        },
    })

    client, _ := kgo.NewClient(kgo.SeedBrokers(brokers...))
    return &SafeKafkaProducer{client: client, breaker: cb}
}

func (p *SafeKafkaProducer) Produce(ctx context.Context, record *kgo.Record) error {
    _, err := p.breaker.Execute(func() (interface{}, error) {
        return nil, p.client.ProduceSync(ctx, record).FirstErr()
    })
    if errors.Is(err, gobreaker.ErrOpenState) {
        // Kafka is down → use fallback (local queue, HTTP, etc.)
        return p.fallback(record)
    }
    return err
}

func (p *SafeKafkaProducer) fallback(r *kgo.Record) error {
    // Write to local disk queue or synchronous DB for later retry
    return writeToLocalBuffer(r)
}
```

Q89. How do you implement RabbitMQ with OpenTelemetry tracing in Go?

Difficulty: Hard

```
import (
    amqp "github.com/rabbitmq/amqp091-go"
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/propagation"
)

func publishWithTrace(ctx context.Context, ch *amqp.Channel, body []byte) error {
    tracer := otel.Tracer("rabbitmq")
    ctx, span := tracer.Start(ctx, "rabbitmq.publish")
    defer span.End()

    // Inject trace context into AMQP headers
```

```

headers := make(amqp.Table)
propagator := otel.GetTextMapPropagator()
propagator.Inject(ctx, propagation.MapCarrier(map[string]string{
    // will be set as: "traceparent", "tracestate" in headers
}))

return ch.Publish("orders", "order.created", false, false,
    amqp.Publishing{
        Body:    body,
        Headers: headers,
    })
}

func consumeWithTrace(d amqp.Delivery) {
    // Extract trace context from AMQP headers
    carrier := make(propagation.MapCarrier)
    for k, v := range d.Headers {
        if s, ok := v.(string); ok { carrier[k] = s }
    }
    ctx := otel.GetTextMapPropagator().Extract(context.Background(), carrier)
    ctx, span := otel.Tracer("rabbitmq").Start(ctx, "rabbitmq.consume")
    defer span.End()
    processMessage(ctx, d.Body)
}

```

Q90. How do you implement a reliable message publisher in Go (Kafka + outbox)?

Difficulty: Hard

```

type Publisher struct {
    db          *pgxpool.Pool
    producer    *kgo.Client
}

// Atomic: write to DB + outbox in one transaction
func (p *Publisher) Publish(ctx context.Context, tx pgx.Tx, event *Event) error {
    data, _ := json.Marshal(event)
    _, err := tx.Exec(ctx,
        `INSERT INTO outbox (id, event_type, payload, created_at)
        VALUES ($1, $2, $3, NOW())`,
        event.ID, event.Type, data)
    return err
}

// Background: flush outbox to Kafka
func (p *Publisher) StartOutboxFlusher(ctx context.Context) {
    for {
        select {
        case <-ctx.Done(): return
        case <-time.After(100 * time.Millisecond): p.flush(ctx)
        }
    }
}

func (p *Publisher) flush(ctx context.Context) {
    rows, _ := p.db.Query(ctx,
        `SELECT id, event_type, payload FROM outbox
        WHERE published_at IS NULL ORDER BY id LIMIT 100 FOR UPDATE SKIP LOCKED`)
    defer rows.Close()
}

```

```

var records []*kgo.Record
var ids []string
for rows.Next() {
    var id, evtType string; var payload []byte
    rows.Scan(&id, &evtType, &payload)
    ids = append(ids, id)
    records = append(records, &kgo.Record{Topic: evtType, Key: []byte(id), Value: payload})
}
if len(records) == 0 { return }

if err := p.producer.ProduceSync(ctx, records...).FirstErr(); err != nil { return }
p.db.Exec(ctx, "UPDATE outbox SET published_at=NOW() WHERE id=ANY($1)", ids)
}

```

Q91. What are the key differences when choosing Kafka vs RabbitMQ for a specific system?

Difficulty:
Medium

Decision Framework:

Question 1: Do consumers need to replay messages?

Yes → Kafka (log retention)

No → Either works

Question 2: What's the expected throughput?

>100K msg/sec → Kafka

<50K msg/sec → Either (RabbitMQ handles ~100K with quorum queues)

Question 3: Do you need complex message routing?

Yes (content-based, topic wildcards) → RabbitMQ exchanges

No (route by key/partition) → Kafka

Question 4: Is exactly-once semantics required?

Yes → Kafka (transactional producer) or RabbitMQ with idempotent consumers

At-least-once with idempotent consumers → Both work well

Question 5: Do you need request-reply (RPC) over messaging?

Yes → RabbitMQ (correlation-id pattern is natural)

No → Either

Question 6: Are you in AWS ecosystem?

Yes → SQS/SNS may be simpler (serverless, no infra)

Anywhere → Kafka or RabbitMQ

Typical recommendation:

Event-driven microservices + high scale: Kafka

Task queues + worker pools: RabbitMQ or SQS

Simple pub/sub: RabbitMQ or Redis Pub/Sub

Stream processing + analytics: Kafka + Flink/Kafka Streams

Master these 100 questions and you'll handle any Kafka/RabbitMQ SDE2 interview. Key areas: Kafka internals (partitions, ISR, EOS), RabbitMQ routing (exchanges, DLX, quorum queues), messaging patterns (saga, outbox, DLQ), and Go integration. ■

Section 4 — Advanced Kafka & RMQ (Q92–Q150)

Q92. What is Kafka transaction API?**Difficulty: Hard**

```
// Transactions: atomic produce across multiple partitions/topics
// Use case: exactly-once processing (read-process-write)

// Producer config
props.put("transactional.id", "order-processor-1"); // unique per producer
props.put("enable.idempotence", "true"); // required for transactions
props.put("acks", "all");

producer.initTransactions();

try {
    producer.beginTransaction();

    // Process: consume from input, produce to output
    ConsumerRecords<String, String> records = consumer.poll(Duration.ofSeconds(1));
    for (ConsumerRecord<String, String> record : records) {
        // Produce to output topic
        producer.send(new ProducerRecord<>("orders-processed", processOrder(record.value())));
    }

    // Atomically commit offsets + produce (exactly-once)
    producer.sendOffsetsToTransaction(
        getOffsets(records), // current consumer offsets
        consumer.groupMetadata()
    );
    producer.commitTransaction();

} catch (Exception e) {
    producer.abortTransaction();
    throw e;
}

// Go: franz-go transactional producer
cl, _ := kgo.NewClient(
    kgo.TransactionalID("processor-1"),
    kgo.RecordPartitioner(kgo.StickyKeyPartitioner(nil)),
)

if err := cl.BeginTransaction(); err != nil { panic(err) }

cl.Produce(ctx, &kgo.Record{Topic: "output", Value: []byte("processed")}, nil)

if err := cl.EndTransaction(ctx, kgo.TryCommit); err != nil {
    cl.EndTransaction(ctx, kgo.TryAbort)
}
```

Q93. What is Kafka schema evolution?**Difficulty: Hard**

Schema evolution: changing message format without breaking consumers

Schema Registry (Confluent):

Central store for Avro/Protobuf/JSON Schema

Producers register schema, get schema ID

Message format: [magic byte][4-byte schema ID][payload]

Consumer uses schema ID to deserialize

Compatibility modes:

BACKWARD: new schema can read data from previous schema
(consumers upgrade first, then producers)
Add optional fields, remove optional fields

FORWARD: previous schema can read data from new schema
(producers upgrade first, then consumers)
Add fields with defaults

FULL: both backward and forward compatible
Only add/remove optional fields with defaults

NONE: no compatibility check (dangerous in production)

Avro example:

v1: {name, email}
v2: {name, email, age (default: 0)} ← backward compatible
v3: {name, email} ← backward compatible (remove optional field)

Bad evolution:

Rename field (not compatible)
Change field type (int → string)
Remove required field

In protobuf:

Don't reuse field numbers
Add new fields (always optional in proto3)
Use reserved keyword for removed fields

Q94. What is Kafka topic partitioning strategy?

Difficulty: Hard

Partition count decision:

Max parallelism = number of partitions
1 partition → 1 consumer max
N partitions → N consumers max

Rules of thumb:

Start: $\max(\text{target throughput} / \text{partition throughput}, \text{consumer count} \times 2)$
Throughput per partition: ~10MB/s write, ~30MB/s read
Don't go too high: each partition = file handles + memory overhead

Example: need 100MB/s throughput, consumers = 20
 $100 / 10 = 10$ partitions for throughput
20 consumers → 20 partitions minimum
Choose: 24 partitions (round number, 2 replicas per broker if 12 brokers)

Partition key selection:

customer_id: related messages co-located (session data)
order_id: random distribution if IDs are sequential
random (null key): uniform distribution across partitions

Hot partition problem:

Key with very high traffic (celebrity user, top merchant)
→ one partition overwhelmed
Solutions:

- Compound key: customer_id + random_suffix
- Custom partitioner: special handling for hot keys
- Repartition to another topic with better key

Changing partitions: NOT trivial

Adding partitions: existing messages stay in old partitions
 Key routing changes → same key may go to different partition
 If order matters per key: very disruptive
 Plan partition count carefully upfront

Q95. What is Kafka consumer offset management?

Difficulty: Hard

```
// Auto commit (default, easiest):
// enable.auto.commit=true, auto.commit.interval.ms=5000
// Risk: messages committed before fully processed (loss on crash)

// Manual commit: commit only after processing
reader := kafka.NewReader(kafka.ReaderConfig{
    Brokers:      []string{"localhost:9092"},
    GroupID:      "order-processor",
    Topic:        "orders",
    CommitInterval: 0, // disable auto-commit
})

for {
    msg, err := reader.FetchMessage(ctx) // fetch without marking
    if err != nil { break }

    if err := processOrder(msg); err != nil {
        // Don't commit → message will be re-processed
        log.Printf("error processing: %v", err)
        continue
    }

    // Commit only after successful processing
    if err := reader.CommitMessages(ctx, msg); err != nil {
        log.Printf("commit failed: %v", err)
    }
}

// Offset reset policies:
// auto.offset.reset=earliest: start from beginning (new consumer group)
// auto.offset.reset=latest: start from now (skip old messages)
// auto.offset.reset=none: throw exception if no committed offset

// Check consumer lag:
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
    --group order-processor --describe
// LAG column: messages behind
```

Q96. What is Kafka Streams stateful operations?

Difficulty: Hard

```
// Kafka Streams: stream processing with state stores
StreamsBuilder builder = new StreamsBuilder();

KStream<String, Order> orders = builder.stream("orders");
```

```
// Stateful: count orders per customer
KTable<String, Long> orderCounts = orders
    .groupByKey()
    .count(Materialized.as("order-counts-store"));

// Windowed aggregation: orders per customer per hour
KTable<Windowed<String>, Long> hourlyOrders = orders
    .groupByKey()
    .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofHours(1)))
    .count();

// Joins: enrich stream with table data
KTable<String, Customer> customers = builder.table("customers");

KStream<String, EnrichedOrder> enriched = orders
    .join(customers,
        (order, customer) -> enrich(order, customer),
        Joined.with(Serdes.String(), orderSerde, customerSerde));

// State stores:
// RocksDB (default): persistent, good for large state
// In-memory: faster, smaller state
// Changelog topic: automatic backup to Kafka for fault tolerance
// Recovery: rebuild state from changelog on restart

// Interactive queries: read state stores directly
ReadOnlyKeyValueStore<String, Long> store =
    streams.store(StoreQueryParameters.fromNameAndType(
        "order-counts-store", QueryableStoreTypes.keyValueStore()));
Long count = store.get(customerId);
```

Q97. What is RabbitMQ shovel and federation?

Difficulty: Hard

Shovel: move messages between queues/exchanges (same or different brokers)

Use case: migrate data, cross-datacenter forwarding, archive

```
# Static shovel (permanent)
rabbitmq-plugins enable rabbitmq_shovel
```

```
rabbitmqctl set_parameter shovel my-shovel \
'{"src-protocol": "amqp091",
  "src-uri": "amqp://localhost",
  "src-queue": "source-queue",
  "dest-protocol": "amqp091",
  "dest-uri": "amqp://remote-broker",
  "dest-queue": "dest-queue"}'
```

Federation: distribute exchanges across brokers

Use case: geographically distributed consumers

Messages follow consumers (pulled, not pushed)

```
rabbitmq-plugins enable rabbitmq_federation
rabbitmq-plugins enable rabbitmq_federation_management
```

```
# Upstream definition
```

```
rabbitmqctl set_parameter federation-upstream my-upstream \
'{"uri": "amqp://upstream-broker", "prefetch-count": 10}'
```

```
# Policy: federate specific exchanges
rabbitmqctl set_policy federation-all "^federated\." \
'{"federation-upstream-set": "all"}' --apply-to exchanges
```

Shovel vs Federation:

```
Shovel: copy messages (both brokers process same message)
Federation: messages pulled to where consumers are (efficient)
Shovel: point-to-point
Federation: dynamic, topology-aware
```

Q98. What is RabbitMQ dead letter exchange (DLX)?

Difficulty: Hard

```
import pika

# DLX setup: where failed messages go
channel.exchange_declare(exchange='dead-letters', exchange_type='direct')
channel.queue_declare(queue='dead-letter-queue')
channel.queue_bind(queue='dead-letter-queue', exchange='dead-letters', routing_key='dead')

# Main queue with DLX configured
args = {
    'x-dead-letter-exchange': 'dead-letters',
    'x-dead-letter-routing-key': 'dead',
    'x-message-ttl': 30000, # 30s TTL → also sends to DLX on expiry
    'x-max-length': 1000,   # when full, oldest go to DLX
}
channel.queue_declare(queue='orders', arguments=args)

# When message becomes dead:
# 1. Rejected with requeue=False: channel.basic_reject(tag, requeue=False)
# 2. Nacked with requeue=False: channel.basic_nack(tag, requeue=False)
# 3. TTL expired (x-message-ttl)
# 4. Queue max-length exceeded

# DLX message headers (added automatically):
# x-death: list of death records (queue, reason, count, time, exchange, routing-keys)
# Useful for: knowing how many times tried, from which queue

# DLX routing:
# Use routing key to route to different DL queues per reason:
# reason=rejected → queue: dlq-rejected
# reason=expired → queue: dlq-expired
```

Q99. What is RabbitMQ quorum queues?

Difficulty: Hard

```
Quorum queues: replicated, durable (replaced mirrored queues in RMQ 3.8+)
Based on Raft consensus algorithm
Leader: handles all reads/writes
Followers: replicate leader's log
Quorum: majority of nodes must agree before message confirmed
```

Advantages over classic mirrored:

```
Safer: Raft prevents split-brain
Faster recovery: follower immediately takes over
Data safety: no message loss on network partition
```

Configuration:

```

rabbitmqctl add_vhost /production

# Declare quorum queue
channel.queue_declare(
    queue='orders',
    durable=True,
    arguments={
        'x-queue-type': 'quorum',
        'x-quorum-initial-group-size': 3, # replicate to 3 nodes
    }
)

```

Feature differences from classic:

- No priority queues
- No per-message TTL (queue-level TTL only)
- No non-durable mode
- No poison message handling (manual with delivery-limit)

Poison message handling:

x-delivery-limit: N → after N deliveries, message discarded to DLX
 args = {'x-queue-type': 'quorum', 'x-delivery-limit': 3}
 Equivalent to SQS maxReceiveCount + DLQ

Q100. What is Kafka Connect and source/sink connectors?

Difficulty: Hard

Kafka Connect: scalable data integration framework

Source connector: external system → Kafka topic

Sink connector: Kafka topic → external system

Architecture:

Connect workers (distributed or standalone)

Workers run connectors (source or sink)

Tasks: parallel units of work within a connector

Common connectors:

Source: Debezium (CDC), JDBC, S3, MongoDB, HTTP

Sink: JDBC, Elasticsearch, S3, BigQuery, Redis

Debezium (CDC source connector):

Reads DB WAL (transaction log)

PostgreSQL: logical replication slots

MySQL: binlog

MongoDB: oplog

config:

```

"connector.class": "io.debezium.connector.postgresql.PostgresConnector"
"database.hostname": "postgres"
"database.port": "5432"
"database.user": "debezium"
"database.dbname": "production"
"plugin.name": "pgoutput"
"slot.name": "debezium"
"table.include.list": "public.orders,public.users"
"topic.prefix": "dbserver1"

```

REST API:

POST /connectors # create connector

```
GET /connectors      # list connectors
PUT /connectors/{name}/pause
DELETE /connectors/{name}
GET /connectors/{name}/status # running, failed, paused
```

Go alternative: Bentbentley (redpanda-data/benthos) for lightweight ETL

Q101. ### Q101–Q150: Advanced Kafka/RMQ Questions

Difficulty:
Medium

Q102. What is Kafka log compaction?

Difficulty:
Medium

Log compaction: retain only latest value per key
Normal: retain all messages up to retention.bytes or retention.ms
Compacted: keep only most recent record per key

Use case: materialized views, CDC (latest state per entity)
orders topic: order_id=42 → status:NEW, status:PROCESSING, status:SHIPPED
After compaction: order_id=42 → status:SHIPPED (only latest)

Config:

```
cleanup.policy=compact (or compact,delete for both)
min.cleanable.dirty.ratio=0.5
segment.ms=86400000 # 24h before compaction eligible
```

Tombstone records:

```
Value=null → delete key from compacted log
After grace period (delete.retention.ms), key purged completely
```

Use for:

- User preferences (latest settings per user)
- Product catalog (latest price/info per product)
- Cache invalidation events
- Event sourcing read models

Q103. What is Kafka consumer group rebalancing?

Difficulty:
Medium

Rebalancing: redistribute partitions among consumers
Triggered by:

- Consumer joins group
- Consumer leaves/crashes (session.timeout.ms = 10s default)
- Partition count changes
- Topic subscription changes

Rebalancing protocols:

- Eager (default): all consumers stop consuming, give up partitions
→ Stop-the-world (processing pauses during rebalance)
- Cooperative (incremental): partition-by-partition reassignment
→ Only affected partitions moved, others continue
Config: partition.assignment.strategy=CooperativeStickyAssignor

Static membership (reducing rebalances):

```
group.instance.id: unique per consumer (persistent member ID)
```

Consumer restarts within `session.timeout.ms` → no rebalance!
Use for: rolling restarts, deploy without downtime

Rebalance listener:

`onPartitionsRevoked`: commit offsets before losing partitions
`onPartitionsAssigned`: reset state for newly assigned partitions

Minimize rebalances:

Increase `session.timeout.ms` (but slow failure detection)
Increase `max.poll.interval.ms` (allow slow processing)
Use static membership (`group.instance.id`)
Process quickly within `max.poll.interval.ms`

Q104. What is Kafka ISR (In-Sync Replicas)?

Difficulty:
Medium

ISR: subset of replicas that are "caught up" with leader
Replica is in-sync if: last fetch within `replica.lag.time.max.ms` (10s default)

`acks=all`: wait for all ISR to acknowledge (not all replicas)
If `ISR = {leader, replica1}` (`replica2` fell behind)
`acks=all` waits for leader + `replica1` only

`min.insync.replicas` (broker/topic config):
Minimum ISR required to accept writes
`min.insync.replicas=2, replication.factor=3`
→ At least 2 replicas must be in sync
→ If only 1 in ISR → producer gets `NotEnoughReplicasException`
→ Prevents data loss at cost of availability

Unclean leader election:
`unclean.leader.election.enable=false` (default, recommended)
→ Leader can only be elected from ISR
→ If all ISR fail: topic becomes unavailable (no data loss)

`unclean.leader.election.enable=true`:
→ Out-of-sync replica can become leader
→ Message loss possible (replica was behind)
→ More available, less durable
→ Use only for: metrics/logs where loss is acceptable

Q105. What is Kafka MirrorMaker 2?

Difficulty:
Medium

MirrorMaker 2 (MM2): active-active/active-passive Kafka replication

Active-passive (DR):

Primary cluster → MM2 → Secondary cluster
Replicated topics: prefixed with source cluster alias
`source.orders` → dest cluster as: `primary.orders`
Consumer group offsets replicated (for failover)

Active-active (geo-distribution):

Cluster A ↔ Cluster B (bidirectional replication)
Cycle prevention: don't replicate back what was replicated
Read locally, write to home cluster
Use case: users in different regions, data sovereignty

Config:

```
clusters = us-east, eu-west
us-east.bootstrap.servers = kafka-us:9092
eu-west.bootstrap.servers = kafka-eu:9092
us-east->eu-west.enabled = true
us-east->eu-west.topics = orders, payments
us-east->eu-west.groups = order-processor
```

MirrorMaker 2 vs Confluent Replicator:

MM2: open source, Apache Kafka project
Replicator: Confluent commercial, more features, better monitoring

Offset translation:

MM2 maintains offset mapping between clusters
Use MirrorClient to translate offsets for consumer failover

Q106. What is RabbitMQ shovel vs Kafka MirrorMaker?**Difficulty:**
Medium**RabbitMQ Shovel:**

Moves messages from source queue/exchange to destination
Single point-to-point connection
Good for: migration, simple forwarding
No ordering guarantee across queues

Kafka MirrorMaker 2:

Replicates topics between Kafka clusters
Maintains ordering per partition
Consumer group offset replication
Multi-cluster topology (active-active, active-passive)
Good for: DR, geo-distribution, data locality

Key differences:

Shovel: queue → queue (point-to-point, any AMQP broker)
MM2: topic → topic (Kafka only, full partition replication)

Shovel: simple, no ordering guarantee
MM2: ordered per partition, offset translation

When shovel-like feature needed in Kafka:

Kafka Streams: consume from source, produce to target (with transformation)
Kafka Connect: source connector → sink connector

Q107. What is Kafka producer compression?**Difficulty:**
Medium

Compression reduces: network bandwidth, broker storage
Cost: CPU on producer (compress) and consumer (decompress)

Algorithms:

lz4: best performance, moderate compression (recommended)
snappy: good performance, moderate compression (Google)
gzip: best compression, slowest (use for large payloads)
zstd: best balance (compression ratio + speed), Kafka 2.1+

Config:

```
compression.type=lz4 (producer-side)
```


Broker can override: `compression.type=producer` (respect producer)
 Or force: `compression.type=gzip` (recompress if needed)

Benchmark (typical):

JSON 1KB per message, 10K messages:
 Uncompressed: 10MB
 lz4: ~4MB (60% reduction), 0.3ms overhead
 snappy: ~4MB, 0.5ms overhead
 gzip: ~2MB (80% reduction), 2ms overhead
 zstd: ~3MB (70% reduction), 0.8ms overhead

Batch compression:

Compression applied per batch (not per record)
 Larger batches → better compression ratio
 Increase: `linger.ms` (wait for batch) + `batch.size`

`linger.ms=10` → wait 10ms before sending (larger batches)
`batch.size=65536` → 64KB batch target

Q108. What is Kafka record ordering guarantees?

Difficulty:
Medium

Ordering guarantees:

Per partition: STRICT ordering

All records with same partition → strict FIFO

Across partitions: NO ordering

record A to partition 0 and record B to partition 1
 Consumer may see B before A

Per key ordering:

Same key → same partition (deterministic hash)
 All records for `customer_id=42` → partition 3 (always)
 Ordering guaranteed for given key

Enable.idempotence + max.in.flight.requests.per.connection:

Default: `max.in.flight=5` (5 parallel in-flight requests)
 If batch 1 fails and batch 2 succeeds → batch 1 retried → OUT OF ORDER

Fix: `max.in.flight.requests.per.connection=1` (one at a time, slow)
 Or: `enable.idempotence=true` (sequence numbers prevent reordering)

`enable.idempotence=true` automatically sets:

`acks=all`
`max.in.flight.requests.per.connection=5` (with reordering prevention)
`retries=Integer.MAX_VALUE`

FIFO with Kafka:

Single partition topic (no parallelism)
 FIFO queue per key (use separate topic per entity)
 Kafka Streams: rekey to guarantee ordering

Q109. What is Kafka consumer lag monitoring?

Difficulty:
Medium

// Consumer lag: messages produced but not yet consumed

```
// Critical metric: indicates processing delay

// Prometheus JMX exporter (Java)
// kafka.consumer:type=consumer-fetch-manager-metrics
// records-lag-max, records-lag, fetch-rate

// kafka-consumer-groups CLI
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group order-processor --describe
// Output: GROUP, TOPIC, PARTITION, CURRENT-OFFSET, LOG-END-OFFSET, LAG, CONSUMER-ID

// Go: calculate lag programmatically
func calculateLag(ctx context.Context, client kgo.Client, group, topic string) (int64, error) {
    offsets, err := kadm.NewClient(&client).ListConsumerGroupOffsets(ctx, group)
    if err != nil { return 0, err }

    endOffsets, err := kadm.NewClient(&client).ListEndOffsets(ctx, topic)
    if err != nil { return 0, err }

    var totalLag int64
    offsets.Each(func(offset kadm.OffsetResponse) {
        end, _ := endOffsets.Lookup(offset.Topic, offset.Partition)
        totalLag += end.Offset - offset.Offset.Offset
    })
    return totalLag, nil
}

// Alert thresholds:
// Warning: lag > 1000 messages
// Critical: lag > 10000 messages
// Or: use ApproximateAgeOfOldestMessage equivalent
//      check timestamp of oldest unprocessed message
```

Q110. What is RabbitMQ vs Kafka for microservices?

Difficulty:
Medium

Choose RabbitMQ when:

- Complex routing (topic exchange, header exchange)
- Per-message acknowledgment and requeue
- Mix of message patterns (queues + pub-sub)
- Lower latency required (microseconds)
- Messages should be deleted after processing
- Flexible routing with binding rules
- Small-medium scale (< 10K msg/sec)

Choose Kafka when:

- High throughput (100K+ msg/sec per broker)
- Message replay / event sourcing needed
- Multiple consumers on same data stream
- Long retention (days to weeks)
- Ordered processing per key
- Audit log, CDC use case
- Real-time analytics pipeline

Hybrid (common):

- Kafka: event backbone (orders, payments, user events)
- RabbitMQ: internal task queues (email, notifications, jobs)

Pattern: Kafka → (consumer service) → RabbitMQ → worker

Feature mapping:

Exchange ≈ Kafka topic

Queue ≈ Consumer group + partition

Binding/routing key ≈ no equivalent (use topic-per-type)

DLX ≈ Kafka DLT (dead letter topic, via Streams)

Message TTL ≈ Kafka log retention

Q111. ### Q110–Q150: Final Kafka/RMQ Topics

Difficulty:
Medium

Q	Topic
Q110	Kafka partition assignment strategies (RangeAssignor, StickyAssignor)
Q111	Kafka segment files and index structure
Q112	Kafka controller election (KRaft vs ZooKeeper)
Q113	RabbitMQ memory and disk alarms
Q114	RabbitMQ connection and channel limits
Q115	Kafka producer interceptors for tracing
Q116	Kafka consumer interceptors for metrics
Q117	RabbitMQ plugins (management, prometheus, shovel)
Q118	Kafka REST Proxy (Confluent)
Q119	Schema Registry API and compatibility check
Q120	Kafka Tiered Storage
Q121	RabbitMQ stream plugin (Kafka-like streams)
Q122	Kafka ACLs and security
Q123	RabbitMQ OAuth2 authentication
Q124	Kafka SASL/SCRAM authentication in Go
Q125	Kafka inter-broker TLS
Q126	RabbitMQ network partition handling
Q127	Kafka partition leadership balance
Q128	Consumer group coordinator
Q129	Kafka cluster rolling upgrade
Q130	RabbitMQ rolling upgrade
Q131	Kafka monitoring with Prometheus + Grafana
Q132	RabbitMQ Prometheus metrics
Q133	Kafka performance tuning guide
Q134	RabbitMQ performance tuning guide
Q135	Kafka disaster recovery playbook
Q136	Go franz-go advanced configuration

Q137	Go stadway/amqp reconnect pattern
Q138	Kafka consumer group parallel processing in Go
Q139	Idempotent consumer pattern in Go
Q140	Saga orchestration with Kafka
Q141	CQRS with Kafka event sourcing
Q142	Debezium CDC patterns
Q143	Kafka Streams testing
Q144	kafka-go vs franz-go performance
Q145	RabbitMQ MQTT plugin
Q146	Kafka WebSocket bridge
Q147	Exactly-once semantics comparison
Q148	Message ordering in distributed systems
Q149	Kafka vs Pulsar vs NATS comparison
Q150	SDE2 Kafka/RMQ interview checklist

Extended Questions (Q112–Q150)

Q112. What is Kafka consumer group rebalancing?

Difficulty: Hard

Rebalancing: redistribution of partitions among consumer group members

Triggered by: member join, member leave, member crash, partition count change

Types (Kafka 2.4+):

Eager rebalance (original):

- All members stop consuming
- All partitions unassigned
- Reassigned in round 2
- Downtime during rebalance!

Cooperative rebalance (incremental):

- Only affected partitions revoked/assigned
- Unaffected members keep consuming
- No stop-the-world!
- Enable: `partition.assignment.strategy = CooperativeStickyAssignor`

Strategies:

- RangeAssignor: assigns contiguous partitions per topic
- RoundRobinAssignor: round-robin across all partitions
- StickyAssignor: minimize partition movement on rebalance
- CooperativeStickyAssignor: cooperative + sticky (best)

Minimize rebalancing:

- Increase `session.timeout.ms` (default 45s)
- Increase `max.poll.interval.ms` (default 300s)
- Increase `heartbeat.interval.ms` (default 3s, should be $< \text{session.timeout}/3$)
- Use static membership: `group.instance.id` (rolling restarts don't rebalance!)

Q113. What is Kafka log compaction?**Difficulty: Hard**

Log compaction: retain only LATEST value per key (vs time/size based retention)

Use: changelog topics, event sourcing snapshots, database change CDC

How it works:

Cleaner thread scans log segments

Keeps: latest value per key

Tombstone: null value → key deleted after retention.ms

Clean vs dirty segments:

Clean: compacted, no duplicate keys

Dirty: uncompact (recent writes)

Head: active write segment

Configuration:

```
cleanup.policy = compact           # enable compaction
cleanup.policy = compact,delete    # compact + delete old data
min.cleanable.dirty.ratio = 0.5    # clean when 50% dirty
delete.retention.ms = 86400000     # keep tombstones 24h
segment.ms = 604800000             # 7-day segments
```

Use case: user profile topic

key=user_id, value=profile_json

After compaction: one entry per user_id (latest state)

Consumer rebuilds: user_id → latest profile (state store)

Guarantee: any consumer reading from beginning sees latest value per key

Not suitable: append-only event log where all events matter

Q114. What is Kafka Streams vs Kafka consumer API?**Difficulty: Hard**

Kafka Consumer API:

Raw access to consume, process, produce

You handle: state management, exactly-once, fault tolerance

Use: simple consumers, stateless processing, custom logic

Kafka Streams:

High-level DSL + Processor API

Built-in: windowing, aggregations, joins, state stores, exactly-once

Fault tolerant: state backed by Kafka topics (changelogs)

No separate cluster needed (runs inside application)

Kafka Streams DSL:

KStream: unbounded stream of records

KTable: changelog stream → materialized to state store

GlobalKTable: replicated to every instance

Example (word count):

```
KStream<String, String> textLines = builder.stream("input");
KTable<String, Long> wordCounts = textLines
    .flatMapValues(v -> Arrays.asList(v.split("\\s+")))
    .groupBy((k, v) -> v)
    .count(Materialized.as("word-counts"));
wordCounts.toStream().to("output");
```

vs Apache Flink:

Kafka Streams: embedded in application, simpler ops

Flink: separate cluster, more powerful (complex CEP, SQL, ML)

Use Kafka Streams for: moderate complexity, Java apps

Use Flink for: complex event processing, large scale

Q115. What is Kafka producer idempotence?

Difficulty: Hard

```
// Idempotent producer: exactly-once delivery per partition
// Prevents: duplicate messages on producer retry

// Enable idempotence
props.put("enable.idempotence", "true")
// This also sets:
// acks = all
// max.in.flight.requests.per.connection = 5
// retries = MAX_INT

// Producer sequence numbers:
// Each message has: producer_id + sequence_number
// Broker deduplicates: if sequence already seen → ignore (don't write again)
// Producer_id regenerated on restart → no cross-session dedup

// Go (franz-go):
client, _ := kgo.NewClient(
    kgo.RecordPartitioner(kgo.StickyKeyPartitioner(nil)),
    kgo.RequiredAcks(kgo.AllISRacks()),
    kgo.DisableIdempotentWrite(false), // enabled by default in franz-go
)

// Limitations:
// Idempotence: per producer session (restart loses idempotence)
// For true cross-session: use transactions
// Scope: single partition only (not cross-partition atomic)

// Check: broker logs "ProducerId" assignment
```

Q116. What is Kafka transactions (exactly-once semantics)?

Difficulty: Hard

```
// Kafka transactions: atomic write to multiple partitions + offsets
// EOS (Exactly-Once Semantics): read-process-write without duplicates

// Producer side
props.put("transactional.id", "order-processor-1") // unique per instance
props.put("enable.idempotence", "true")           // required

producer.initTransactions()

// Consume-transform-produce loop
for {
    records := consumer.poll()

    producer.beginTransaction()
    for _, r := range records {
        result := transform(r)
        producer.send("output-topic", result)
    }
}
```

```

    }

    // Commit offsets INSIDE transaction (atomic with messages)
    producer.sendOffsetsToTransaction(offsets, consumer.groupMetadata())
    producer.commitTransaction()
    // If crash after commitTransaction: nothing re-processed
    // If crash before: transaction aborted on next startup
}

// Consumer side: must read only committed messages
props.put("isolation.level", "read_committed") // skip aborted messages

// Overhead: ~10-15% throughput reduction
// Use when: payment processing, order fulfillment, financial data

```

Q117. What is Kafka consumer offset management?

Difficulty:
Medium

```

// Offsets: consumer's position in partition
// Committed offset: last processed position (stored in __consumer_offsets topic)

// Auto commit (default, risky):
// enable.auto.commit = true
// auto.commit.interval.ms = 5000
// Risk: crash after poll but before processing = messages lost!

// Manual commit (recommended):
// enable.auto.commit = false

// Commit after processing (at-least-once)
msgs, _ := consumer.FetchMessages(ctx)
for _, msg := range msgs {
    process(msg)
}
consumer.CommitMessages(ctx, msgs...) // commit after all processed

// franz-go: manual commit
client.PollRecords(ctx, 100)
records := results.Records()
for _, r := range records {
    process(r)
}
client.CommitRecords(ctx, records...)

// Offset reset policies (when no committed offset):
// auto.offset.reset = latest → start from newest (default)
// auto.offset.reset = earliest → start from beginning

// Seek to specific offset (reprocessing):
consumer.Seek(tp, offset)

// Consumer lag: committed_offset - latest_offset
// Monitor with: kafka-consumer-groups.sh --describe
// Or: Burrow, LinkedIn's consumer lag monitor

```

Q118. What is Kafka topic configuration for production?**Difficulty:**
Medium

```
# Create topic with production settings
kafka-topics.sh --create \
  --topic orders \
  --partitions 12 \           # throughput × expected consumers
  --replication-factor 3 \    # HA: survive 2 broker failures
  --config retention.ms=604800000 \    # 7 days retention
  --config retention.bytes=107374182400 \    # 100GB max
  --config cleanup.policy=delete \    # delete old segments
  --config min.insync.replicas=2 \    # acks=all needs 2 ISR
  --config compression.type=lz4 \    # compress (better: producer-side)
  --config message.max.bytes=1048576 # 1MB max message

# Producer matching config
acks = all                # wait for all ISR
min.insync.replicas = 2    # topic-level (must match topic config)
compression.type = lz4     # fast, good ratio
linger.ms = 5              # batch for 5ms
batch.size = 65536         # 64KB batch

# Partition count rules:
# desired_throughput / single_partition_throughput
# Number of consumers in group (can't have more consumers than partitions)
# AWS MSK recommendation: partitions ≤ 4000 per broker

# Monitor topic health:
kafka-topics.sh --describe --topic orders --bootstrap-server :9092
# Check: Leader, Replicas, ISR (should match Replicas)
# ISR < Replicas = replica lagging (alert!)
```

Q119. What is Kafka schema registry?**Difficulty: Hard**

```
// Schema Registry: centralized schema management
// Prevents: producer/consumer schema mismatch breaking consumers
// Avro schemas stored: subject = topic-key or topic-value

// Schema evolution rules:
// BACKWARD (default): new schema can read data written with old schema
// FORWARD: old schema can read data written with new schema
// FULL: both backward + forward compatible

// Avro schema (user.avsc)
{
  "type": "record",
  "name": "User",
  "fields": [
    {"name": "id", "type": "long"},
    {"name": "name", "type": "string"},
    {"name": "email", "type": ["null", "string"], "default": null} // optional field
  ]
}

// Producer with schema registry (confluent-kafka-go)
p, _ := kafka.NewProducer(&kafka.ConfigMap{
  "bootstrap.servers": "broker:9092",
```



```

    "schema.registry.url": "http://schema-registry:8081",
  })
  serializer, _ := avro.NewSpecificSerializer(client, serde.ValueSerde, avro.NewSerializerConfig())

  // Consumer: deserialize with schema from registry
  deserializer, _ := avro.NewSpecificDeserializer(client, serde.ValueSerde, avro.NewDeserializerConfig())
  user := &User{}
  deserializer.DeserializeInto(topic, msg.Value, user)

```

Q120. What is Kafka MirrorMaker 2 (MM2)?

Difficulty: Hard

MirrorMaker 2: cross-cluster replication for Kafka

Uses Kafka Connect internally

Use cases:

- Active-passive DR: replicate prod cluster to standby
- Active-active: bidirectional replication (geo-distributed)
- Aggregation: multiple clusters → one central cluster
- Migration: cluster A → cluster B (zero-downtime)

Topic renaming:

- source.cluster.topic → target prefixed: source.topic
- Configurable: replication.policy.class

Offset translation:

- Source offset ≠ Target offset (different segments)
- MM2 maintains offset mapping in `__consumer_offsets`
- Consumer failover: translate offsets automatically

mm2.properties:

```

clusters = source, target
source.bootstrap.servers = source:9092
target.bootstrap.servers = target:9092
source->target.enabled = true
source->target.topics = orders.*
source->target.groups = app-consumers.*
replication.factor = 3
emit.checkpoints.interval.seconds = 60

```

Active-active pitfalls:

- Write cycles: message replicated back to source
- Fix: cycle detection via `source.cluster.alias` header
- Use: Kafka version ≥ 2.4 (has built-in cycle detection)

Q121. What is Kafka broker internals?

Difficulty: Hard

Kafka broker: server that stores and serves partitions

Log storage:

- Partition = ordered, immutable log
- Stored as: segments (each ~1GB by default)
- Segment files: `.log` (messages), `.index` (offset→position), `.timeindex`

Log segment lifecycle:

- Active segment: current writes go here
- Closed segments: immutable, eligible for retention cleanup

Retention: delete segments older than retention.ms / larger than retention.bytes

Fetch path:

Consumer requests: topic, partition, offset, max_bytes
 Broker: find segment, find offset in index, read from disk
 Zero-copy: sendfile() syscall (kernel → socket, no user-space copy)
 Result: very fast even for disk-based storage

Write path:

Producer → leader partition
 Leader writes to .log file
 ISR replicas fetch from leader (same fetch mechanism as consumers)
 Leader tracks ISR: replica is in-sync if lag < replica.lag.time.max.ms (10s)

Controller:

One broker elected as controller (via ZooKeeper or KRaft)
 Manages: leader elections, ISR updates, topic creation/deletion
 KRaft (Kafka 3.3+): controller embedded in Kafka (no ZooKeeper!)

Q122. What is RabbitMQ quorum queues vs classic queues?

Difficulty: Hard

Classic Queues (legacy):

In-memory or disk mirrored
 Mirroring via policies: ha-mode, ha-params
 Not truly replicated: synchronous replication optional
 Issues: split-brain, sync lag, slow mirroring after failure

Quorum Queues (RabbitMQ 3.8+, recommended):

Based on Raft consensus protocol
 Replicated by default across N nodes (quorum = N/2+1)
 Properties:
 At-least-once delivery (stronger guarantee)
 No data loss on majority survival
 Leader failover: automatic in <10 seconds
 No pause-minority issues

Create quorum queue:

```
rabbitmqadmin declare queue name=orders durable=true \
  arguments='{"x-queue-type": "quorum", "x-quorum-initial-group-size": 3}'
```

In Go (amqp091-go):

```
ch.QueueDeclare("orders", true, false, false, false, amqp.Table{
  "x-queue-type": "quorum",
})
```

Quorum queue limitations:

No priority queues
 No lazy mode
 Slightly higher memory (Raft state)
 Consumer acknowledgment required (no auto-ack)

Migration: rabbitmq-upgrade quorum_queue_migration

Q123. What is RabbitMQ shovel and federation?

Difficulty: Hard

Shovel: move messages from source to destination (within or cross-cluster)

Reliable: acknowledges at source only after successful delivery
 Use: cross-datacenter, cluster migration

```
rabbitmq-plugins enable rabbitmq_shovel
```

Configuration:

```
Source: amqp://source-cluster/orders
Destination: amqp://target-cluster/orders
Delete-after: never (permanent) or queue-length 0 (drain)
```

Federation: link exchanges/queues across clusters (loose coupling)
 Publisher → Exchange (cluster A) → federated to cluster B → Consumer
 Uses: geographically distributed consumers

```
rabbitmq-plugins enable rabbitmq_federation
```

Policy:

```
rabbitmqctl set_policy federate-orders "^orders$" \
'{"federation-upstream-set": "all"}' --apply-to exchanges
```

Shovel vs Federation:

```
Shovel: moves messages (deleted from source after ack)
Federation: distributes messages (source keeps copy if local consumers)
Shovel: point-to-point
Federation: fan-out to multiple clusters
```

Q124. What is Kafka consumer group lag monitoring?

Difficulty:
Medium

```
# Check consumer lag
kafka-consumer-groups.sh --bootstrap-server broker:9092 \
  --describe --group my-consumer-group

# Output:
# TOPIC    PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  CONSUMER-ID
# orders   0          12345           12350           5    consumer-1
# orders   1          11100           11100           0    consumer-2
# orders   2          9999            10100           101   consumer-3 ← high lag!

# LAG = LOG-END-OFFSET - CURRENT-OFFSET
# LAG = 0: consumer is caught up
# LAG growing: consumer can't keep up (scale out or optimize)

# Burrow (LinkedIn): dedicated Kafka consumer lag monitor
# Features: lag trend analysis, alerting, multiple Kafka clusters

# Prometheus monitoring (Kafka Exporter):
kafka_consumergroup_lag{group="my-group", partition="0", topic="orders"} 5

# Alerting:
# Alert if lag > 1000 AND lag growing for 5+ minutes

# In Go: calculate lag in consumer
func getLag(client sarama.Client, group, topic string, partition int32) (int64, error) {
    committed, _ := client.GetOffset(topic, partition, sarama.OffsetNewest)
    // Note: sarama doesn't directly get committed offset
    // Use admin client or kafka-consumer-groups API
```

```
    return 0, nil
}
```

Q125. What is Kafka dead letter queue pattern?

Difficulty:
Medium

```
// Kafka DLQ: route failed messages to separate topic for investigation

func consumeWithDLQ(client *kgo.Client, dlqClient *kgo.Client) {
    for {
        fetches := client.PollFetches(context.Background())
        fetches.EachRecord(func(r *kgo.Record) {
            var retries int

            for retries < maxRetries {
                if err := processRecord(r); err == nil {
                    client.CommitRecords(context.Background(), r)
                    return
                }
                retries++
                time.Sleep(backoff(retries))
            }

            // All retries exhausted → send to DLQ
            dlqRecord := &kgo.Record{
                Topic: r.Topic + ".dlq",
                Value: r.Value,
                Headers: append(r.Headers,
                    kgo.RecordHeader{Key: "original-topic", Value: []byte(r.Topic)},
                    kgo.RecordHeader{Key: "original-partition", Value: []byte(fmt.Sprintf(r.Partition))},
                    kgo.RecordHeader{Key: "error", Value: []byte("max retries exceeded")},
                    kgo.RecordHeader{Key: "timestamp", Value: []byte(time.Now().String())},
                ),
            }
            dlqClient.ProduceSync(context.Background(), dlqRecord)
            client.CommitRecords(context.Background(), r)
        })
    }
}
```

Q126. What is Kafka and Debezium for CDC?

Difficulty: Hard

```
# Debezium: CDC (Change Data Capture) connector for Kafka Connect
# Reads database WAL → publishes change events to Kafka

# Postgres connector config (connector.json)
{
    "name": "postgres-connector",
    "config": {
        "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
        "database.hostname": "postgres",
        "database.port": "5432",
        "database.user": "debezium",
        "database.password": "secret",
        "database.dbname": "mydb",
        "database.server.name": "myserver",
        "table.include.list": "public.orders,public.users",
```

```

    "plugin.name": "pgoutput",
    "slot.name": "debezium",
    "publication.name": "dbz_publication",
    "transforms": "route",
    "transforms.route.type": "org.apache.kafka.connect.transforms.ReplaceField$Value",
    "key.converter": "io.confluent.kafka.serializers.KafkaAvroSerializer",
    "value.converter": "io.confluent.kafka.serializers.KafkaAvroSerializer"
  }
}

# Output topics: myserver.public.orders, myserver.public.users
# Each message: {before: {...}, after: {...}, op: "c"|"u"|"d"|"r"}
# op: c=create, u=update, d=delete, r=read (snapshot)

# Use cases:
# Cache invalidation: detect DB changes → update Redis
# Search indexing: detect changes → update Elasticsearch
# Audit log: capture all changes to separate DB
# Event sourcing: DB changes become domain events

```

Q127. What is Kafka performance tuning?

Difficulty: Hard

Producer tuning (throughput):

```

batch.size = 65536          # 64KB batch (default 16KB)
linger.ms = 5               # wait 5ms to fill batch
compression.type = lz4      # fast + good ratio
buffer.memory = 67108864    # 64MB buffer
max.in.flight.requests.per.connection = 5 # concurrent requests

```

Producer tuning (latency):

```

linger.ms = 0               # no batching delay
batch.size = 1              # immediate send
compression.type = none     # no compression overhead
acks = 1                    # don't wait for ISR

```

Consumer tuning (throughput):

```

fetch.min.bytes = 1048576   # 1MB min fetch (wait for batch)
fetch.max.wait.ms = 500     # wait 500ms if not enough data
max.poll.records = 500      # batch size per poll
fetch.max.bytes = 52428800  # 50MB per fetch

```

Consumer tuning (latency):

```

fetch.min.bytes = 1         # return immediately if any data
fetch.max.wait.ms = 0       # no wait
max.poll.records = 1        # process one at a time

```

Broker tuning:

```

num.io.threads = 8          # I/O threads
num.network.threads = 3     # network threads
socket.send.buffer.bytes = 102400
log.flush.interval.messages = 10000 # batch fsync

```

Q128. What is RabbitMQ connection and channel management in Go?

Difficulty: Hard

```

// Connection: TCP connection to RabbitMQ (expensive, keep alive)
// Channel: virtual connection inside Connection (lightweight, many per conn)

```

```
// Reconnection logic
type RabbitMQ struct {
    conn    *amqp.Connection
    channel *amqp.Channel
    done    chan struct{}
}

func (r *RabbitMQ) connect(url string) error {
    var err error
    r.conn, err = amqp.DialConfig(url, amqp.Config{
        Heartbeat: 10 * time.Second,
        Locale:    "en_US",
    })
    if err != nil { return err }

    r.channel, err = r.conn.Channel()
    return err
}

func (r *RabbitMQ) reconnectLoop(url string) {
    for {
        select {
            case <-r.done: return
            case err := <-r.conn.NotifyClose(make(chan *amqp.Error)):
                log.Printf("connection closed: %v, reconnecting...", err)
                for {
                    time.Sleep(5 * time.Second)
                    if err := r.connect(url); err == nil {
                        log.Println("reconnected successfully")
                        break
                    }
                }
                log.Printf("reconnect failed: %v", err)
            }
        }
    }
}

// Best practice:
// 1 connection per application
// 1 channel per goroutine (channels are not goroutine-safe!)
// Monitor connection/channel closures
```

Q129. What is Kafka Connect?

Difficulty:
Medium

Kafka Connect: framework for connecting Kafka with external systems

Source connectors: external system → Kafka

Sink connectors: Kafka → external system

Distributed mode: runs as cluster, fault-tolerant

Popular connectors:

Debezium (source): PostgreSQL, MySQL, MongoDB → Kafka (CDC)

JDBC source: any JDBC database → Kafka

S3 sink: Kafka → AWS S3 (data lake)

Elasticsearch sink: Kafka → Elasticsearch

JDBC sink: Kafka → database

Redis sink: Kafka → Redis

REST API management:

```

POST /connectors      # create connector
GET  /connectors      # list connectors
GET  /connectors/{name}/status # check health
PUT  /connectors/{name}/pause  # pause
POST /connectors/{name}/restart # restart
DELETE /connectors/{name}      # delete

```

Transforms (SMT - Single Message Transform):

```

Route messages to different topics
Filter messages by condition
Add/remove fields
Rename fields

```

vs writing custom consumer:

```

Kafka Connect: declarative, no code, standardized
Custom: full control, complex transformations
Use Connect for: standard integrations (DB, S3, ES)
Use custom for: complex business logic

```

Q130. What is RabbitMQ priority queues?**Difficulty:**
Medium

```

// Priority queue: higher priority messages consumed first
// Max priority: 1-255 (high memory usage with high values)
// Recommended: 1-5 priority levels

// Declare priority queue
ch.QueueDeclare("tasks", true, false, false, false, amqp.Table{
    "x-max-priority": int32(5), // max priority level
})

// Publish with priority
ch.PublishWithContext(ctx, "", "tasks", false, false,
    amqp.Publishing{
        Priority:    5, // high priority
        ContentType: "application/json",
        Body:        body,
    })

// Priority 5 messages consumed before priority 1
// Mixed delivery: some lower priority interleaved (not strictly FIFO within level)

// Consumer: normal declaration, priority handled by RabbitMQ
msgs, _ := ch.Consume("tasks", "", false, false, false, false, nil)

// Warning: priority queues consume more memory
// Don't use priority > 10 (quadratic memory growth)
// Alternative: separate queues per priority level (more predictable)
ch.QueueDeclare("tasks.high", ...)
ch.QueueDeclare("tasks.normal", ...)
ch.QueueDeclare("tasks.low", ...)
// Consumer: try high first, fallback to normal, then low

```

Q131. ### Q131–Q150: Final Kafka/RMQ Questions**Difficulty:**
Medium

Q	Topic
Q131	Kafka topic naming conventions
Q132	RabbitMQ management plugin and monitoring
Q133	Kafka KRaft mode (no ZooKeeper)
Q134	Consumer back-pressure handling
Q135	Kafka topic partitioning strategy
Q136	RabbitMQ exchange bindings and routing
Q137	Kafka message key design
Q138	RabbitMQ shovel for message migration
Q139	Kafka producer callbacks and error handling
Q140	RabbitMQ connection recovery in Go (amqp091)
Q141	Kafka compacted topic read semantics
Q142	Message ordering in distributed systems
Q143	Kafka cluster sizing for production
Q144	RabbitMQ memory and disk alarms
Q145	Kafka and multi-datacenter deployment
Q146	Saga pattern with Kafka vs RabbitMQ
Q147	Kafka message encryption (TLS + application level)
Q148	RabbitMQ vs SQS for job queues
Q149	Kafka consumer crash recovery flow
Q150	Messaging production checklist

Q132. What is Kafka topic naming conventions?

Difficulty:
Medium

Best practices for Kafka topic names:

Format: <domain>.<entity>.<event-type>

Examples:

```
orders.order.placed
payments.payment.processed
users.user.registered
inventory.product.stock-updated
```

Rules:

- Use lowercase and dots (.) as separators
- Avoid special chars: /, \, spaces, #, %
- Max length: 249 characters
- Don't start with underscore (internal Kafka topics use __)

Internal topics:

```
__consumer_offsets  – consumer group offset storage
__transaction_state – transaction coordinator state
__schemas           – Confluent Schema Registry
```

Environment prefix:


```
dev.orders.order.placed
staging.orders.order.placed
prod.orders.order.placed
```

Versioning:

```
orders.order.placed.v2 (when schema changes incompatibly)
```

Q133. What is Kafka KRaft mode (no ZooKeeper)?

Difficulty:
Medium

```
# KRaft: Kafka Raft Metadata mode (Kafka 3.3+ production-ready)
# Eliminates ZooKeeper dependency

# Benefits:
# - Simpler operations (one system instead of two)
# - Faster startup and failover
# - Supports 10x more partitions (millions vs 200K)
# - Single security model

# Configure KRaft (server.properties):
process.roles=broker,controller # combined mode (dev)
# or: process.roles=broker      # broker-only
# or: process.roles=controller  # controller-only (prod)

node.id=1
controller.quorum.voters=1@controller1:9093,2@controller2:9093,3@controller3:9093
listeners=PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
controller.listener.names=CONTROLLER

# Generate cluster ID and format storage:
kafka-storage.sh random-uuid # generates cluster UUID
kafka-storage.sh format -t <UUID> -c server.properties

# Start without ZooKeeper:
kafka-server-start.sh server.properties
```

Q134. What is Kafka consumer back-pressure handling?

Difficulty:
Medium

```
// Back-pressure: consumer slower than producer → lag grows
// Solutions:

// 1. Increase consumer instances (scale out, up to partition count)
// 2. Optimize processing (async I/O, batch DB writes)
// 3. Rate limit processing (don't overwhelm downstream)

// Controlled processing rate
func consumeWithRateLimit(client *kgo.Client, rps int) {
    limiter := rate.NewLimiter(rate.Limit(rps), rps)

    for {
        fetches := client.PollFetches(context.Background())
        fetches.EachRecord(func(r *kgo.Record) {
            limiter.Wait(context.Background()) // respect rate limit
            process(r)
        })
        client.CommitUncommittedOffsets(context.Background())
    }
}
```

```

    }
}

// 4. Batch processing (reduce per-message overhead)
func batchProcess(records []*kgo.Record) error {
    // Batch insert to DB: 1 query for 100 records vs 100 queries
    return db.BulkInsert(records)
}

// 5. Pause/resume consumption
client.PauseFetchPartitions(map[string][]int32{"topic": {0, 1}})
// ... downstream recovers ...
client.ResumeFetchPartitions(map[string][]int32{"topic": {0, 1}})

```

Q135. What is Kafka message ordering guarantees?

Difficulty:
Medium

Within a partition: strict ordering guaranteed
 Messages published to partition 0: 1, 2, 3 → consumed as 1, 2, 3

Across partitions: NO ordering
 Partition 0: [A, B]
 Partition 1: [C, D]
 Consumer may see: A, C, B, D or any interleaving

Key-based ordering:
 Same key → same partition (via hash)
 All events for user_id=42 → same partition → ordered

```

producer.Send("orders", key="order-123", value=OrderCreated)
producer.Send("orders", key="order-123", value=OrderPaid)
producer.Send("orders", key="order-123", value=OrderShipped)
→ always consumed in this order

```

Gotcha: producer retries with max.in.flight > 1
 Without idempotence: retry can reorder messages
 Fix: enable.idempotence=true (default in modern Kafka clients)

FIFO across partitions (workaround):
 Single partition (max 1 consumer, low throughput)
 External sequencer (database sequence number)
 Sagas with compensating transactions

Q136. What is RabbitMQ monitoring and management?

Difficulty:
Medium

```

# Management plugin (built-in)
rabbitmq-plugins enable rabbitmq_management

# Web UI: http://localhost:15672 (admin/admin)
# REST API: http://localhost:15672/api/

# Key metrics to monitor:
# - Memory usage (alert > 60% of memory watermark)
# - Disk space (alert < 50MB free)
# - Queue depth (messages ready)
# - Consumer count per queue (should be > 0!)

```

```
# - Message rates (publish/deliver/ack)
# - Connection/channel count

# CLI monitoring
rabbitmqctl status
rabbitmqctl list_queues name messages consumers
rabbitmqctl list_exchanges name type
rabbitmqctl list_connections

# Prometheus integration
rabbitmq-plugins enable rabbitmq_prometheus
# Scrape: http://localhost:15692/metrics

# Alerts:
# rabbitmq_queue_messages_ready > 10000 (backlog growing)
# rabbitmq_queue_consumers == 0 AND messages > 0 (consumers dead!)
# rabbitmq_node_mem_used / rabbitmq_node_mem_limit > 0.8
```

Q137. What is Kafka producer error handling?

Difficulty:
Medium

```
// franz-go: synchronous produce with error
result := client.ProduceSync(ctx, &kgo.Record{
    Topic: "orders",
    Key:   []byte(orderID),
    Value: payload,
})
if err := result.FirstErr(); err != nil {
    // Retriable errors (handled by client automatically):
    // - LeaderNotAvailable, NotLeaderOrFollower
    // - RequestTimedOut (with retries configured)

    // Non-retriable errors (your responsibility):
    // - MessageTooLarge → compress or split
    // - InvalidTopicException → topic doesn't exist
    // - AuthorizationException → check ACLs

    if errors.Is(err, kerr.MessageSizeTooLarge) {
        log.Printf("message too large: %d bytes", len(payload))
        return sendToFallback(payload)
    }
    return fmt.Errorf("produce failed: %w", err)
}

// Async produce with callback
client.Produce(ctx, record, func(r *kgo.Record, err error) {
    if err != nil {
        metrics.ProduceErrors.Inc()
        log.Printf("async produce failed: %v", err)
        // Consider: retry queue, DLQ, alerting
    }
})
```

Q138. What is RabbitMQ exchange types deep dive?

Difficulty:
Medium

Direct exchange:

Route by exact routing key match

```
rabbitmqadmin declare exchange name=direct_logs type=direct
```

Binding: queue=error_queue, routing_key=error

Publish: routing_key=error → goes to error_queue only

Fanout exchange:

Broadcast to ALL bound queues (ignores routing key)

Use: notifications, cache invalidation, audit logging

All consumers receive every message

Topic exchange:

Pattern matching: * (one word), # (zero or more words)

"order.created.us-east" matches "order.created.*" and "order.#"

Use: selective subscriptions to event streams

Headers exchange:

Route by message headers (not routing key)

More flexible than topic, slightly slower

```
headers={"x-match": "all", "format": "pdf", "type": "report"}
```

Default exchange:

Empty name ""

Direct routing: routing key = queue name

Used by simple apps (AMQP default)

Dead Letter Exchange (DLX):

Failed messages → DLX → DLQ

Configure: x-dead-letter-exchange, x-dead-letter-routing-key

Inspect failed messages without blocking main queue

Q139. What is Kafka consumer crash recovery?

Difficulty:
Medium

Crash recovery flow:

1. Consumer crashes mid-processing:

- Last committed offset: 100
- Last processed: 105 (not committed before crash)
- On restart: starts from 100 → reprocesses 101-105
- Result: at-least-once delivery (duplicates possible)

2. Rebalancing triggered:

- Other consumers notice via heartbeat timeout (session.timeout.ms)
- Group coordinator reassigns partitions
- New consumer starts from last committed offset

3. Make consumers idempotent:

- Unique message ID in payload
- Upsert: INSERT ... ON CONFLICT DO UPDATE
- Deduplication table: record processed message IDs

4. Reduce window of duplicates:

- Commit frequently (after each message or small batch)
- Trade-off: more commits = more Kafka overhead

5. Exactly-once (Kafka Transactions):

- Consumer reads + produces within transaction

- Commit offsets inside transaction
- On crash: transaction aborted → clean restart from last committed offset

Q140. What is Kafka and schema evolution strategies?

Difficulty:
Medium

Schema evolution: changing message format without breaking consumers

BACKWARD compatible change (consumers can read old + new):

- Add optional field with default
- Remove field (consumers ignore unknown fields)
- Add required field without default
- Rename field
- Change field type

FULL compatible (recommended):

New schema can read old data (backward)
Old schema can read new data (forward)
Only: add/remove optional fields with defaults

Breaking changes (avoid or use new topic):

Field rename → create new field, deprecate old
Type change → new field, transform in consumer
Required → optional → never: new topic + migration

Avro evolution example:

v1: {id: long, name: string}
v2: {id: long, name: string, email: string (default: "")} ← backward compat
v3: {id: long, name: string, email: string, phone: string (default: null)} ← OK

Version deployment order:

Backward: deploy consumers first, then producers
Forward: deploy producers first, then consumers
Full: deploy either order safely

Q141. What is Kafka cluster sizing for production?

Difficulty:
Medium

Sizing formula:

Throughput:

Write throughput: $\text{producers_TPS} \times \text{avg_message_bytes}$
Replication: $\text{throughput} \times \text{replication_factor}$
Total: $\text{throughput} \times (1 + \text{replication_factor})$

Storage:

$\text{daily_write} \times \text{retention_days} \times \text{replication_factor}$
Example: $100\text{GB/day} \times 7 \text{ days} \times 3 = 2.1\text{TB per cluster}$

Partitions per broker:

Recommended: ≤ 4000 partitions per broker (Kafka guideline)
KRaft mode: supports millions of partitions

Broker specs:

RAM: 64GB (OS page cache is critical for Kafka performance)
CPU: 12-24 cores (mostly for compression/decompression)
Disk: 12TB NVMe SSD per broker, dedicated (no OS sharing)

Network: 10Gbps (replicas compete with consumers)

Rule of thumb per broker:

10 Gbps network → 1 GB/s write (with 3x replication = 300MB/s user data)

For high throughput: multiple brokers, more partitions

JVM settings:

-Xms6g -Xmx6g (fixed heap, avoid GC pauses)

-XX:+UseG1GC -XX:MaxGCPauseMillis=20

Q142. What is RabbitMQ vs Kafka for job queues?

Difficulty:
Medium

RabbitMQ for job queues:

- Native work queue semantics (pull-based consumers)
- Per-message TTL, priority, DLX built-in
- Acknowledgment: requeue on nack (automatic retry)
- Complex routing (topic/headers exchange)
- Low latency (<1ms with persistence off)
- Easy to set up
- No message replay
- Smaller ecosystem

Best for: task queues, job processing, workflow

Kafka for job queues:

- Replay: reprocess failed jobs from any point
- Long retention (audit trail of all jobs)
- Very high throughput
- Multiple consumer groups (different workers for same jobs)
- Consumer offset management complexity
- No priority queues
- No per-message TTL
- Requires external retry logic

Best for: high-volume jobs, audit needed, stream processing

Decision:

Priority / DLQ / complex routing → RabbitMQ

Replay / audit / massive throughput → Kafka

Both → separate queues for different job types

Q143. What is Kafka sarama vs franz-go vs confluent-kafka-go?

Difficulty:
Medium

sarama (Shopify):

Pure Go, most mature, large community

Cons: complex API, some performance issues

github.com/IBM/sarama (transferred from Shopify)

franz-go (kgo):

Pure Go, modern API, highest performance

Benchmarks: 2-3x faster than sarama

Features: transactions, cooperative rebalancing, compression

github.com/twmb/franz-go

confluent-kafka-go:

CGO wrapper around librdkafka (C library)

Pros: battle-tested C library, full feature parity

Cons: CGO → harder to cross-compile, requires C libs
github.com/confluentinc/confluent-kafka-go

go-kafka (segmentio):
 Pure Go, clean API
github.com/segmentio/kafka-go
 Simpler than sarama, less features than franz-go

Recommendation (2025):
 New projects: franz-go (best performance, modern API)
 Existing sarama: migrate to IBM/sarama (active fork)
 Need Confluent features: confluent-kafka-go

```
// franz-go example:
client, _ := kgo.NewClient(
    kgo.SeedBrokers("broker1:9092", "broker2:9092"),
    kgo.ConsumerGroup("my-group"),
    kgo.ConsumeTopics("orders"),
)
```

Q144. What is Kafka exactly-once in Go?

Difficulty:
Medium

```
// Exactly-once: transactions API
// consumer reads → process → producer writes + commit offsets atomically

// Producer with transactions
client, _ := kgo.NewClient(
    kgo.SeedBrokers("broker:9092"),
    kgo.TransactionalID("processor-1"), // enables transactions
    kgo.RequiredAcks(kgo.AllISRacks()),
)

ctx := context.Background()
client.BeginTransaction()

// Consume
fetches := consumer.PollFetches(ctx)
fetches.EachRecord(func(r *kgo.Record) {
    result := transform(r)

    // Produce within transaction
    client.Produce(ctx, &kgo.Record{
        Topic: "output-topic",
        Value: result,
    }, nil)
})

// Commit offsets and transaction atomically
if err := client.EndTransaction(ctx, kgo.TryCommit); err != nil {
    client.EndTransaction(ctx, kgo.TryAbort)
}

// Consumer: must read only committed
consumerClient, _ := kgo.NewClient(
    kgo.IsolationLevel(kgo.ReadCommitted()), // skip aborted transactions
)
```

Q145. What is Kafka producer batching internals?**Difficulty:**
Medium

Batching: accumulate records before sending → higher throughput

Configuration:

```
batch.size: max bytes per batch (default 16KB)
linger.ms: wait this long to fill batch (default 0ms = immediate)
```

RecordAccumulator (per partition):

```
Buffer: deque of ProducerBatch objects
Current batch: append records until full or linger.ms expires
Full batch or timer fires → sender thread picks up batch
```

Sender thread:

```
Groups batches by broker (leader)
Sends ProduceRequest: multiple topic-partition batches in one network call
Compression: applied per batch (lz4, snappy, gzip, zstd)
```

Tuning for throughput:

```
linger.ms=5-50 → wait up to 50ms for larger batches
batch.size=65536 → 64KB batches
compression.type=lz4 → fast + good ratio
buffer.memory=67108864 → 64MB total buffer
```

Tuning for latency:

```
linger.ms=0 → send immediately
batch.size=1 → never batch
acks=1 → don't wait for ISR
```

Throughput numbers (tuned):

```
Untuned: ~100K messages/sec
Tuned: 1M+ messages/sec per producer
```

Q146. What is RabbitMQ dead letter exchange pattern in Go?**Difficulty:**
Medium

```
// DLX: route failed messages to dead letter queue for inspection

// Setup: declare main queue with DLX
ch.QueueDeclare("orders", true, false, false, false, amqp.Table{
    "x-dead-letter-exchange": "orders-dlx",
    "x-dead-letter-routing-key": "orders-dead",
    "x-message-ttl": int32(300000), // 5 min TTL → DLX on expire
})

// Declare DLX and DLQ
ch.ExchangeDeclare("orders-dlx", "direct", true, false, false, false, nil)
ch.QueueDeclare("orders-dlq", true, false, false, false, nil)
ch.QueueBind("orders-dlq", "orders-dead", "orders-dlx", false, nil)

// Consumer: Nack without requeue → goes to DLX
msgs, _ := ch.Consume("orders", "", false, false, false, false, nil)
for msg := range msgs {
    if err := process(msg); err != nil {
        // false = don't requeue → triggers DLX
    }
}
```



```

        msg.Nack(false, false)
        continue
    }
    msg.Ack(false)
}

// DLQ consumer: inspect and handle failed messages
dlqMsgs, _ := ch.Consume("orders-dlq", "", false, false, false, false, nil)
for msg := range dlqMsgs {
    log.Printf("DLQ message: %s", msg.Body)
    // Re-publish to main queue after fixing, or alert ops
    msg.Ack(false)
}

```

Q147. What is Kafka compacted topic consumer in Go?

Difficulty:
Medium

```

// Compacted topic: only latest value per key (like a KV store)
// Consumer: replay from beginning to build current state

func buildStateFromCompactedTopic(client *kgo.Client, topic string) (map[string][]byte, error) {
    // Start from beginning
    client.AddConsumeTopics(topic)

    state := make(map[string][]byte)

    // Read until caught up (reach high watermark)
    for {
        fetches := client.PollFetches(context.Background())
        if fetches.IsClientClosed() { break }

        var reachedEnd bool
        fetches.EachPartition(func(p kgo.FetchTopicPartition) {
            for _, r := range p.Records {
                if r.Value == nil {
                    // Tombstone: delete key
                    delete(state, string(r.Key))
                } else {
                    state[string(r.Key)] = r.Value
                }
            }
            // Check if we've reached the end
            if p.HighWatermark <= p.Records[len(p.Records)-1].Offset+1 {
                reachedEnd = true
            }
        })

        if reachedEnd { break }
    }

    return state, nil
}

// Use: config store, user preferences, feature flags stored in Kafka

```

Q148. What is Kafka message key design patterns?

Difficulty:
Medium

Key determines partition assignment: $\text{hash}(\text{key}) \% \text{partitions}$

Design goals:

1. Even distribution (avoid hot partitions)
2. Related events on same partition (ordering)
3. Stable key (don't change partitioning scheme)

Patterns:

Entity ID (most common):

key = user_id / order_id / session_id
 All events for entity X → same partition → ordered
 Good distribution if IDs distributed evenly

Composite key:

key = tenant_id + ":" + entity_id
 Multi-tenant: events per tenant ordered

Round-robin (null key):

key = nil → producer round-robins partitions
 Maximum parallelism, no ordering
 Use: independent events, log aggregation

Time-based (anti-pattern):

key = timestamp → all go to same partition at peak!
 Avoid for high-throughput topics

Geographic:

key = region_id
 Predictable: region X always on partition Y
 Risk: uneven regions = hot partitions

Hash key explicitly:

// If entity ID is sequential (0,1,2,...) → use composite key
 key = fmt.Sprintf("%d", id % numBuckets) // 64 buckets

Q149. What is messaging system production checklist?

Difficulty:
Medium

Kafka Production Checklist:

- Replication factor ≥ 3 for critical topics
- min.insync.replicas = 2 (producer acks=all)
- Consumer group lag monitoring (Burrow or Prometheus)
- DLQ (dead letter topic) for failed messages
- Schema Registry for Avro/Protobuf schemas
- Rack-aware replication (replicas on different AZs)
- Log retention set per topic (not global default)
- Consumer idempotency (handle at-least-once delivery)
- Proper partition count (throughput / partition_throughput)
- Consumer group instance IDs (static membership, fewer rebalances)

RabbitMQ Production Checklist:

- Quorum queues (not classic) for durability
- Publisher confirms enabled (verify delivery)
- Consumer acks (manual acknowledgment)
- DLX configured for each queue
- Message TTL on queues (prevent unbounded growth)

- Connection recovery logic in consumer
- Memory and disk watermarks monitored
- Cluster: ≥ 3 nodes for quorum queue majority
- Shovel/Federation for cross-datacenter
- Management plugin metrics exported to Prometheus

Both:

- Alerts: consumer lag, DLQ depth, broker health
- Runbook for consumer outage
- Load tested before production

Q150. What is event-driven architecture patterns?

Difficulty:
Medium

Patterns for reliable event-driven systems:

Outbox Pattern:

Problem: write to DB + publish to Kafka atomically
Solution: write event to outbox table in same DB transaction
Debezium reads outbox table (CDC) → publishes to Kafka
Guarantee: event published exactly once if DB write succeeds

Saga Pattern:

Long-running distributed transaction across services
Choreography: each service publishes events, others react
Orchestration: central saga orchestrator coordinates steps
Compensating transactions: undo completed steps on failure

Event Sourcing:

Store: sequence of events (not current state)
Replay: rebuild state by replaying events
Snapshot: checkpoint to avoid full replay
Use Kafka compacted topics or event store (EventStoreDB)

CQRS (Command Query Responsibility Segregation):

Write model: commands → events → event store
Read model: events → projections (optimized for queries)
Kafka: event backbone between write and read models

Inbox Pattern (idempotent consumer):

Record consumed message IDs in inbox table
Check inbox before processing (skip duplicates)
Clear inbox entries after N days